Computer Networks 266 (2025) 111330

Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet

SIEVE: Generating a cybersecurity log dataset collection for SIEM event classification

Pierpaolo Artioli [a], Vincenzo Dentamaro [b], Stefano Galantucci [b,*], Alessio Magrì [a], Gianluca Pellegrini [a], Gianfranco Semeraro [b]

[a] Cybersecurity Laboratory, BV TECH S.p.A., Milan, 20123, Italy
[b] University of Bari Aldo Moro, Bari, 70125, Italy

ARTICLE INFO

Dataset link: https://www.cybersuite.it/en/sieve-cybersecurity-log-dataset-collection-forsiem-event-classification/

Keywords:
SIEM
Cybersecurity
Dataset
Log classification
Text classification
Natural Language Processing (NLP)

ABSTRACT

Effective cyber threat monitoring relies on deploying robust Security Information and Event Management (SIEM) systems. SIEM applications receive security events generated by different devices, systems, and applications. They should properly correlate them to identify potential cyber threats based on tactics, techniques, and procedures (TTP), bypassing other security mechanisms (e.g., firewall, IDS, etc.). Given that logs are primarily generated to notify relevant system events and activities in a human-readable format, supervised Natural Language Processing (NLP) techniques could be used to train models that complement conventional parsing methodologies by automatically suggesting event classification into pre-defined categories. Training such models requires a substantial amount of pre-classified (labeled) data of different types to provide the learning patterns and nuances needed to make accurate predictions. Since the number of security event datasets is scarce due to privacy or availability reasons, and the few publicly available ones are often limited in terms of event diversity, number of labels, or simply unfit for the task at hand, an effective synthetic dataset for training SIEM-related machine learning event classification algorithms could be very useful. For these reasons, this paper proposes the generation of a synthetic dataset specifically designed to train SIEM systems for log-type classification. This research paper, starting from an in-depth methodological analysis of the prominent Cybersecurity related datasets available in the literature, introduces SIEVE (Siem Ingesting EVEnts), a synthetic dataset collection built from publicly available log samples using SPICE (Semantic Perturbation and Instantiation for Content Enrichment), a novel text augmentation and perturbation technique. SPICE is shown to be effective in generating realistic logs. Each instance of the dataset collection displays different levels of augmentation. Subsequent performance assessments were conducted through comprehensive benchmarking against various NLP classification models. Tests were conducted by training the classifiers using SIEVE and testing them on both the same SIEVE logs and real logs. The results of the experiments show that the best model among those tested is SVM (MaF1 0.9323 - 0.9737), which maintains its performance with slight degradation, even in tests on real logs (MaF1 0.9477 - 0.9636). BERT, on the other hand, performs better than SVM in most of the tests on SIEVE (MaF1 0.9528 - 0.9730) but does not show robustness when tested on real logs (MaF1 0.8864 - 0.9182).

1. Introduction

In cybersecurity, Security Information and Event Management (SIEM) systems play a pivotal role [1] in protecting digital environments by aggregating and analyzing events generated by a variety of applications and systems. An essential step within the SIEM data processing workflow is event parsing, which converts raw event log messages received into structured data suitable for subsequent analysis; the specific type of log is also derived through a classification process. Natural Language Processing (NLP) techniques are generally employed for the automatic execution of this step [2], which, however, require a substantial amount of properly labeled data [3] to be trained. This is a challenging task because, in the literature, there are currently no public datasets available that can be used to train such a system. Currently, the publicly available textual log datasets do not contain labels for classification or, on the other hand, have binary labels of "normal" versus "anomalous" type; in both cases, therefore, classification of

* Corresponding author.
E-mail addresses: pierpaolo.artioli@bvtech.com (P. Artioli), vincenzo.dentamaro@uniba.it (V. Dentamaro), stefano.galantucci@uniba.it (S. Galantucci), alessio.magri@bvtech.com (A. Magrì), gianluca.pellegrini@bvtech.com (G. Pellegrini), gianfranco.semeraro@uniba.it (G. Semeraro).

https://doi.org/10.1016/j.comnet.2025.111330
Received 1 June 2024; Received in revised form 18 April 2025; Accepted 19 April 2025
Available online 8 May 2025
1389-1286/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

the specific type of event is not provided. This absence results in real difficulty in training, at the outset, SIEM systems to classify logs; thus, cybersecurity experts are forced to rely on commercial solutions, bringing with it the problems of the lack of standardization among different vendors. Hence, there is a need to create datasets that satisfy the above-mentioned characteristics. As noted in [4], collecting real logs poses several issues related to privacy and representativeness of the collected data; therefore, the authors argue that the best solution is the generation of synthetic datasets. This paper introduces SPICE (Semantic Perturbation and Instantiation for Content Enrichment), a methodology that can generate a variety of realistic synthetic logs from real logs. SPICE produces templates corresponding to the structure of each log. The templates are then used, in conjunction with augmentation and perturbation techniques, to balancefully generate synthetic but realistic logs. By using SPICE, this paper proposes a collection of log datasets that meet the above requirements, called SIEVE (SIem ingesting EVEnts). Datasets are generated using logs coming from different public repositories, namely Loghub [5], OSSEC [6], and Public Security Log Sharing Site [7].

The main contributions of this work are the following:

- The proposal of a collection of six different multiclass synthetic datasets built upon publicly available log data, expanded with a novel technique that combines text augmentation and perturbation methodologies called SPICE (Semantic Perturbation and Instantiation for Content Enrichment). Each instance of the dataset displays different augmentation and perturbation entropy levels;
- A comprehensive experimentation on the generated datasets with nine different classification models (Naïve-Bayes, Decision Tree, K-Nearest Neighbors, Logistic Regression, Random Forest, Stochastic Gradient Descent, Support Vector Machine, BERT, and LSTM).

This paper is organized as follows. Section 2 analyzes related work, considering the limitations of current datasets, related techniques of text augmentation and perturbation, and the challenges in generating synthetic data for SIEMs. Section 3 describes the proposed SIEVE collection. Section 4 shows the data-gathering phase and details the selection criteria applied to the original logs to provide SIEVE generation baseline log data. Section 5 details SPICE, the process that generates the SIEVE Datasets starting from baseline log data. Section 6 describes the experiments conducted to evaluate the performance of the algorithms on SIEVE, and the impact of noise introduced by SPICE on model performance. Conclusions are reported in Section 7. The SIEVE datasets are available at the following URL.[1]

2. Related work

This section reviews works from the literature related to the contribution of this paper. Section 2.1 reviews cybersecurity log datasets, discussing methodological differences between SIEVE and these datasets, and pointing out the limitations for which they cannot be used to train a SIEM system. Section 2.2 discusses text augmentation and perturbation techniques. Section 2.3 discusses challenges in synthetic data generation and, more specifically, for SIEM systems.

2.1. Log datasets for cybersecurity

Many datasets are available in the literature for use in cybersecurity, particularly in the context of IDS/IPS and SIEM systems. One of the earliest examples is DARPA 1998 [8], which contains data for numerous attack types on UNIX-based systems; however, this dataset not only fails to match the purposes of this paper, since it does not contain SIEM events, but also contains outdated attacks. CSIC2010 [9] provides HTTP logs with attack-type labels, but the logs are preprocessed into features rather than presented in their raw form. Several datasets containing raw logs exist, such as Harvard's e-commerce Apache logs [10] and Windows Event Log [11], but they lack categorical labels essential for classification tasks. Among the most comprehensive and heterogeneous log sources are Loghub [5], OSSEC [6], and Public Security Log Sharing Site [7]. These collections were selected as the building blocks for SIEVE due to their diverse log types and extensive coverage of security events. However, they differ methodologically from SIEVE in several key aspects: while they provide extensive raw log data, they lack the semantic classification layer that SIEVE introduces. The manual labeling process in SIEVE followed a structured taxonomy specifically designed for SIEM operations, whereas these existing repositories primarily focus on log collection without standardized categorization frameworks. Landauer et al.'s AIT Log Dataset [4] offers synthetic security logs with some overlap to those used in SIEVE, but employs a different synthesis methodology and focuses on attack classification rather than log type categorization. The datasets presented in [2] do provide multiclass labeling related to specific SIEM log types, ranging in size from 16,511 to 34,501 events and containing between 18 and 103 event categories. However, they represent a significant gap in public accessibility, having been built using IBM's proprietary QRadar knowledge of parsed events. This inaccessibility highlights a persistent challenge in the field, as many advanced SIEM datasets remain proprietary due to their development using sensitive customer data. The security dataset landscape remains dominated by collections designed for IDS/IPS systems, such as NSL-KDD [12] (an evolution of KDD99 [13]), CIC-IDS2017 [14], and UNSW-NB15 [15]. These datasets are fundamentally unsuitable for SIEM systems as they contain network traffic data rather than textual logs. Moreover, they classify events based on attack types (normal/anomalous or normal/specific attack) rather than log semantics, which diverges from SIEVE's purpose of semantic classification.

Therefore, there exists a significant methodological gap in publicly available log datasets that SIEVE aims to address. While existing repositories provide valuable raw data, they lack the semantic classification framework necessary for advanced SIEM operations. Available datasets either contain data formats different from raw logs or, where classification exists, it focuses on binary attack detection rather than log type semantics [16]. This limitation is often compounded by accessibility issues, as organizations developing advanced SIEM classification systems typically rely on proprietary, sensitive customer data that cannot be shared with the broader research community. SIEVE addresses these challenges by providing a synthetic, yet realistic dataset that enables log-specific semantic classification starting from raw logs themselves, while maintaining public accessibility and avoiding privacy concerns associated with real-world security data.

2.2. Text augmentation and perturbation techniques

As detailed in the previous section, publicly available event datasets are either unsuitable for multi-class log classification or unclassified while not providing enough diversity for the task. Different text augmentation and perturbation techniques have been analyzed to address these challenges. Contextualized Perturbation for Textual Adversarial Attack, also called CLARE [17], proposes a text perturbation methodology aimed at triggering errors in machine learning models while keeping the results close to the original. After splitting the input text into a list of the most minor meaningful units (tokens), it masks some of them, replacing the missing text with a placeholder that could then be altered with three different contextualized perturbations: Replace, Insert, and Merge, which replace a token, insert a new one, and merge a sequence of two consecutive words (bigram). A Backtranslation technique is proposed in [18], which leverages bidirectional translation models for translating an existing example text input X in language

[1] https://www.cybersuite.it/en/sieve-cybersecurity-log-dataset-collection-forsiem-event-classification/

A into another language B and then translating it back into A to obtain an augmented example Y. Inspired by language comprehension capabilities demonstrated by Large Language Models (LLMs) such as ChatGPT, a novel approach to text data augmentation called AugGPT [19] proposes a sentence rephrasing methodology that, based on a training sample, produces a conceptually similar but semantically distinct output. CheckList [20] is an evaluation methodology for conducting comprehensive behavioral testing of NLP models that offers, among other capabilities, multiple abstractions such as templates, lexicons, and general-purpose perturbations that have been implemented in TextAttack [21] as an augment that alters tokens in a text by combining techniques such as Name Replacement, Location Replacement, Number Alteration, and word Contraction/Extension. Easy Data Augmentation (EDA) [22] proposes a combination of four operations for boosting the performance of text classification tasks, showing particular effectiveness when applied to small starting datasets. Given an input sentence, EDA randomly selects and applies one of the following techniques: Synonim Replacement (SR), Random Insertion (RI), Random Swap (RS), and Random Deletion. Among the above approaches, EDA is undoubtedly the simplest in that it operates without context information. In contrast, the operations performed by CLARE, while similar to those of EDA, are constrained to maintaining the specific context. Even the perturbations introduced by CheckList and TextAttack, while operating at the token level, tend to maintain contextual meaning; Checklist, however, while used in these scenarios, is not designed to increase the amount of data but rather to test the extent to which the models can generalize and handle specific perturbations. Backtranslation maintains context through the process of double semantic translation and, therefore, obtains texts with a more varied vocabulary. AugGPT, like Backtranslation, does not operate by focusing on operations on individual tokens as much as, instead, to generate natural sentences with different conceptual perspectives and syntactic variability. In this paper, EDA has been used as the basic augmentation methodology. This choice arises from the peculiarity of the task at hand: generating logs requires particular methodologies capable of not deflecting from the objective. The logs have distinctive structures, with sparse and technical vocabulary, where articles, conjunctions, and other syntactic forms are almost absent. Therefore, it is necessary to employ methodologies that obtain sentences distant from natural language to obtain realistic logs. Among the models discussed above, tools such as AugGPT and Backtranslation generate unrealistic logs precisely because they aim to generate sentences with syntactic structures similar to natural language. Techniques such as CLARE, CheckList, and TextAttack are concerned with maintaining context, but again, there is a risk of deviating from the naturality of the log; the use of technical language requires that each term has a precise meaning, and, therefore, great care must be taken with any substitution of terms. This form of attention is not profound in methodologies of this type and risks creating significant problems, considering that logs often consist of a few keywords that could be completely transformed, taking on another meaning. EDA, by its simplicity, offers a reasonable basis for generating synthetic logs; however, in our work, the process has been supervised by human experts who defined the specific dictionaries to avoid the issues described above.

2.3. Challenges in synthetic data generation for SIEMs

To the best of our knowledge, this is the first time a synthetic generation process has been applied for creating a SIEM dataset with semantic log-type labeling. Synthetic log generation is typically performed by operating in an automated manner on a testbed and recording the logs. Synthetically generated logs on the testbed [23] tend to be unvarying in normal traffic simulations. On the other hand, as pointed out in [4], capturing real logs poses a significant privacy problem since real logs often contain user-referable information or file content information. Real logs, however, are considered more reliable because they are derived from real usage and not from a simulation. For these reasons, we chose to start from real logs and use text augmentation and perturbation techniques to obtain synthetic data, with a manual classification oriented to SIEM and log typology, rather than IDS/IPS-based binary or attack type-specific classifications. This approach opens several research directions in synthetic dataset generation for SIEM systems. The semantic integrity of logs represents a fundamental challenge: security logs employ specialized technical language with precise meanings that most text augmentation techniques, designed primarily for natural language, struggle to preserve. The heterogeneous nature of SIEM data introduces additional complexity, as logs from diverse sources (network devices, servers, applications) exhibit vastly different formats and vocabularies, requiring specialized transformation rules for each log type. Therefore, precautions have been taken within SPICE to avoid introducing biases, as detailed in Section 5. Temporal patterns in security events present another dimension rarely addressed in synthetic generation. Real security incidents typically manifest as correlated sequences across multiple log sources with specific timing relationships, a pattern difficult to synthetically replicate without sophisticated modeling.

3. SIEVE datasets collection

SIEVE is a collection of six different versions, i.e., six different datasets, named SIEVE_0n with $n \in [0, 5]$. Each dataset comprises 100,000 logs evenly distributed over 30 different classes. Each dataset entry is a pair consisting of the textual log and its class (label). Datasets class structure is shown in Table 1.

SIEVE is synthetically generated from a set of log baselines (Combined Dataset — CDS) obtained from Loghub [5], OSSEC [6] and Public Security Log Sharing Site [7]; the data gathering phase from which the CDS originates is described in Section 4. The process that generates SIEVE from CDS, named *Semantic Perturbation and Instantiation for Content Enrichment* (SPICE), is detailed in Section 5. The six versions of the dataset differ from each other according to the entropy coefficient parameter applied in SPICE, which defines the amount of text augmentation:

- SIEVE_00 is obtained by applying SPICE to the CDS with $p = 0$ (only perturbation without augmentation);
- SIEVE_0x is obtained by applying SPICE to the CDS with $p = 0.x$;

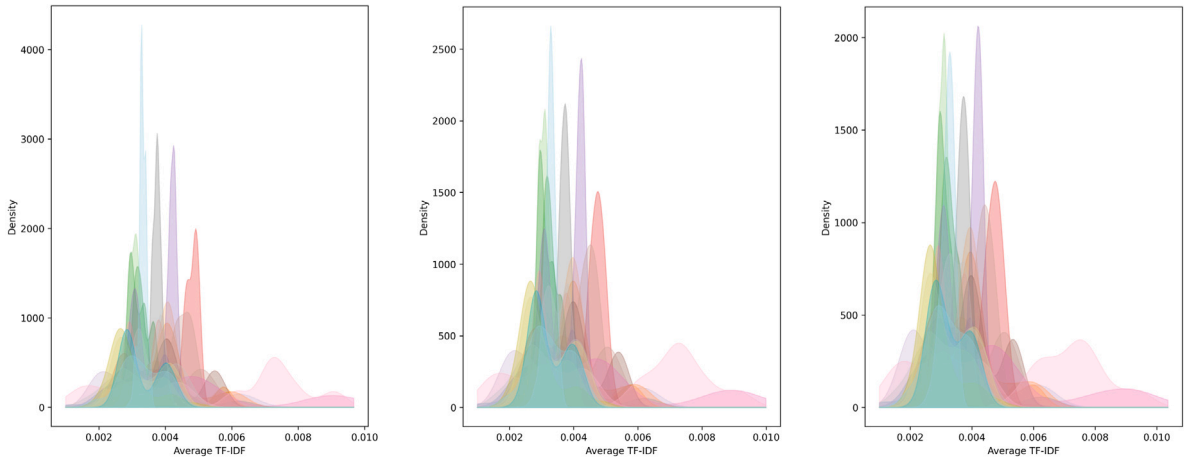
The level of randomness introduced is governed by a parameter called the entropy coefficient p (in other studies, the parameter is often denoted by α), which determines the extent to which transformations are applied to the data. The meaning and the effects of p are detailed in Section 5. As p increases, there is a corresponding increase in the generation of logs that are less similar to the original logs, thus favoring generalization at the expense of log purity; on the other hand, a lower value of p implies that the generated logs will be highly similar to the original logs, implying higher fidelity to the disadvantage of the generalization ability of the model that will be trained. The entropy coefficient p has been set between 0 and 0.5; this is a practical decision that balances the need to increase data and its variability with the preservation of meaningful information. Previous studies in the literature [22,24], including those used as a base for the SPICE algorithm, have reported optimal performance in the same p ranges. Other similar studies [25–27], on the other hand, choose or find as an ideal value one of those belonging to the proposed set. For example, the Easy Data Augmentation (EDA) approach, a key component of the SPICE algorithm, uses $[0.05, 0.1, 0.2, 0.3, 0.4, 0.5]$ as coefficients. This range is the same as the one used in this work, except that the 0.05 value has been substituted for the 0 value; this choice arises from the fact that, in the generation phase, it turned out that the variations introduced are, for the most part, really not significant, so much so that they practically fall into the same case scenarios involving the $p = 0$ value,

Table 1
Class structure of SIEVE datasets. Distribution of class instances versus the combined dataset (CDS) from which SIEVE originates.

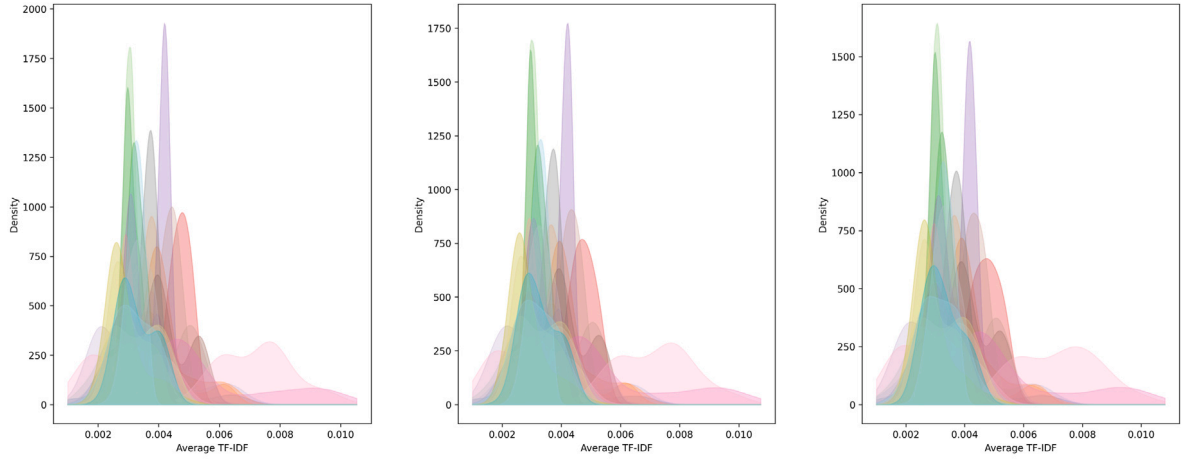
Class #	Class name (Label)	Description (meaning of logs)	Combined dataset		SIEVE datasets	
			Count	%	Count	%
0	authentication-failed	Unsuccessful user access attempts	69	15.68%	3334	3.334%
1	authentication-success	Successful user access attempts	37	8.41%	3334	3.334%
2	connection-closed	Termination of a network or application connection	15	3.41%	3334	3.334%
3	connection-failed	Unsuccessful attempts to establish a network or application connection between entities	13	2.95%	3334	3.334%
4	connection-opened	Successful attempts to establish a connection	16	3.64%	3334	3.334%
5	database-operation	Generic database operation on the schema or data	9	2.05%	3334	3.334%
6	directory-changed	A change in the directory location or path has occurred within a system or application environment	4	0.91%	3334	3.334%
7	directory-created	A directory has been successfully created within a system or application environment	3	0.68%	3334	3.334%
8	directory-deleted	A directory has been successfully deleted from a system or application environment	3	0.68%	3334	3.334%
9	file-action-failure	Attempts to perform generic actions on files resulting in failures within a system or application environment	2	0.45%	3334	3.334%
10	file-deleted	A file has been successfully deleted from a system or application environment	4	0.91%	3333	3.333%
11	file-modification	Operation on file creation or renaming	2	0.45%	3333	3.333%
12	file-read	A file has been successfully read or accessed	3	0.68%	3333	3.333%
13	file-write	A file has been successfully written	3	0.68%	3333	3.333%
14	hardware-monitoring	Encompasses events related to monitoring hardware components within a system or device	22	5.00%	3333	3.333%
15	http-request-failure	Represents instances in which an HTTP (Hypertext Transfer Protocol) request to a server or web application has resulted in failure	3	0.68%	3333	3.333%
16	http-request-success	Represents instances in which an HTTP (Hypertext Transfer Protocol) request to a server or web application has resulted in success.	4	0.91%	3333	3.333%
17	ids-alert	Designates instances where an Intrusion Detection System (IDS) has identified and signaled a potential security breach or malicious activity within a network or system environment.	93	21.14%	3333	3.333%
18	network-filtered	Denotes occurrences where network traffic has undergone filtering processes. potentially by firewalls or security devices	7	1.59%	3333	3.333%
19	network-traffic	Denotes occurrences where allowed network traffic was detected	6	1.36%	3333	3.333%
20	process-ended	A running process within a system or application environment has gracefully terminated or concluded its execution	6	1.36%	3333	3.333%
21	process-error	A running process within a system or application environment encounters an error during the execution	6	1.36%	3333	3.333%
22	process-info	Generic information logged using a process within a system or application environment during its execution	7	1.59%	3333	3.333%
23	process-shutdown	A running process within a system or application environment was forcefully terminated	12	2.73%	3333	3.333%
24	process-started	A new process has been initiated and started execution within a system or application environment	34	7.73%	3333	3.333%
25	system-configuration-changed	Alterations have been made to the configuration settings of the system or application	22	5.00%	3333	3.333%
26	user-creation	New user accounts are established within a system or application environment	8	1.82%	3333	3.333%
27	user-deletion	User accounts are removed or deleted from a system or application environment	5	1.14%	3333	3.333%
28	user-logout	Users successfully terminate their active sessions within a system or application	19	4.32%	3333	3.333%
29	user-session-open	Users initiate new sessions within the system or application environment	3	0.68%	3333	3.333%
		TOTAL	440	100.00%	**100 000**	**100.00%**

i.e., the perturbation only. In fact, for completeness, $p = 0$ is included to allow for comprehensive analysis in the absence of text augmentation. The value 0.05 has also been excluded in the work [24], which uses our same range of values, excluding $p = 0$. Similarly, the process has been tested with $p > 0.5$ values, finding that it tends to generate logs that are significantly different from those from which they originate, thus departing from the purpose of this dataset: in fact, these logs have been found to be very different from the typical structure of the SIEM logs analyzed and could, on the other hand, be misleading for the system, since it would find itself learning about artificial data that have no genuine pertinence to the usually produced logs. The last consideration is likewise reported in [22]. Overall, such an interval allows the algorithm to achieve the desired increment and variability effects without overloading the learning process with noise.

Because the dataset is synthetic, the overall number of logs to be generated must be predefined. The SIEVE log size is defined according to the log classification approach defined in [2]. Referring to the datasets proposed in such work, the *FortiGate* one contains 34,501 samples spanning over 25 different classes. Considering that, as shown in Table 1, the number of classes devised in the CDS (30) is close to *FortiGate* one, the first iteration of the dataset generation test involved the creation of a synthetic dataset comprising 35,000 samples, the closest rounded number to the *FortiGate* size. The dataset generation test is then repeated by increasing the dataset size to find an optimal



(a) SIEVE_00 (b) SIEVE_01 (c) SIEVE_02



(d) SIEVE_03 (e) SIEVE_04 (f) SIEVE_05

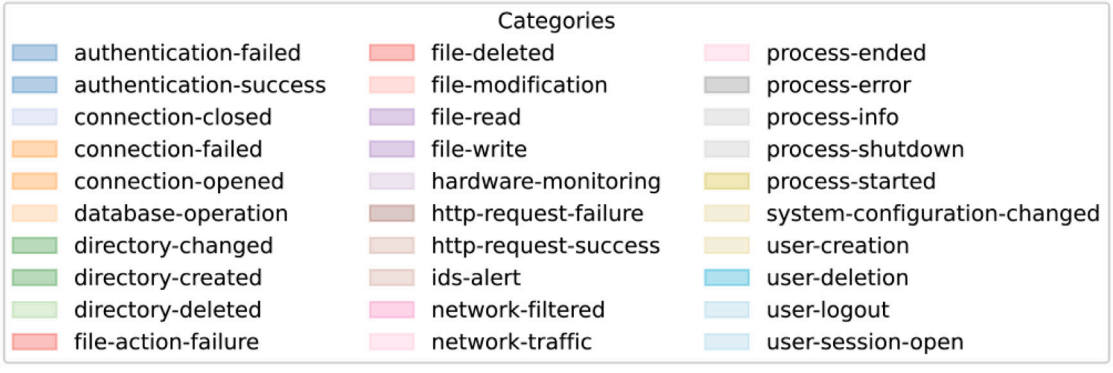


Fig. 1. Kernel density estimation plots of SIEVE. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

size, resulting in negligible performance gains over other benchmarked NLP approaches, as discussed in Section 6.6.

3.1. Dataset analysis

The Kernel Density Estimation (KDE) plots presented in Fig. 1 illustrate the distributions of average TF–IDF values across several categories at varying entropy ratios, specifically from 0 to 0.5. Each subplot is associated with a distinct entropy ratio, providing an opportunity to observe the evolution of these distributions as entropy increases. Notably, across all examined entropy levels, a significant density concentration around TF–IDF values is approximately within the range of 1×10^{-5} to 2×10^{-5}. This observation implies that for a majority of the categories, the TF–IDF values for most log entries are fairly consistent, indicating that the textual features underlying these categories share common attributes. A crucial observation from the plots is the extensive overlap between different categories across all entropy levels. This extensive overlap indicates that numerous categories have similar textual features, which may complicate efforts to distinguish them using only TF–IDF features from naïve classifiers. The overlapping

distributions suggest that the categories are not delineated within the feature space, for this reason, it is indicated the use of strong non linear classifiers such as transformers. As the entropy ratio increases from 0 to 0.5, the KDE plots demonstrate a reduction in peak density alongside a modest broadening of the distributions. This broadening is particularly evident at entropy ratios of 0.4 and 0.5, suggesting that the TF–IDF values exhibit more significant variability. This increased variability could be attributed to including more diverse or potentially noisy features as entropy rises. At lower entropy ratios, such as 0, the higher peak densities indicate that the data is more concentrated around specific TF–IDF values. However, with higher entropy, the data distribution becomes more dispersed, covering a broader spectrum of TF–IDF values. In specific categories, there are noticeable secondary peaks at higher TF–IDF values (e.g., around 4×10^{-5} and above). These secondary peaks, particularly in categories represented by light gray and yellow curves, suggest that these categories might correspond to specific events with unique textual characteristics, resulting in less overlap with other categories. The category associated with the light gray color consistently displays a distinct peak at higher TF–IDF values, indicating that its distribution is notably different from other categories. The KDE plots for entropy ratios between 0.3 and 0.5 reveal more complex and multimodal distributions, indicative of increased heterogeneity within specific categories. This complexity suggests that the feature space becomes more intricate as higher entropy ratios introduce additional complexity, potentially due to the inclusion of noise or less relevant features. The increasing dispersion of TF–IDF values at higher entropy ratios highlights the growing complexity of the feature space, which could either improve model performance by capturing more subtle distinctions or degrade it by introducing noise. The kernel density analysis has the dual benefit of visually depicting the effect of the cumulative perturbations and providing quantitative results with respect to the sensitivity of our synthetic set of data. The KDE curves follow a flatter shape with lower peak heights with an increase in the entropy factor (p), yet the general density profiles—especially the combination of the logarithmic categories—display stability. This finding indicates that while SPICE imparts sufficient variability to allow generalization, the underlying semantic framework of the logs does not degrade. These findings based on the kernel density analysis also enhance the credibility of our synthetic generation technique.

The Principal Component Analysis plots presented in Fig. 2 depict the arrangement of categories across a range of entropy ratios, specifically from 0 to 0.5. These graphical representations provide significant insights into the effectiveness of category separation within the feature space after the dimensionality of TF–IDF vectors is reduced to two principal components. At an entropy ratio of 0, the categories display clearly defined clusters with minimal overlap, signifying that the textual features are highly successful in differentiating between categories. The tightness of these clusters indicates low variance within each category, suggesting a high degree of similarity in the textual features among documents classified under the same category. This observation implies that the dataset quality is robust at this entropy level, characterized by strong internal consistency and distinct category boundaries. As the entropy ratio increases from 0.1 to 0.3, there is a discernible increase in the overlap between certain categories. This expanding overlap suggests a decline in the distinctiveness of the categories as entropy rises, potentially due to the introduction of noise or less relevant features. Concurrently, the clusters become less compact and more dispersed within the PCA space, reflecting greater internal variability within the categories. This increased dispersion may result from the entropy introducing a broader range of textual features, which in turn diminishes the dataset's capacity to maintain distinct category boundaries. At higher entropy ratios, particularly between 0.4 and 0.5, the overlap between categories becomes increasingly pronounced, and the clusters lose their distinctiveness. The merging of some categories suggests a considerable decline in the dataset's quality concerning category differentiation. The deterioration of cohesive cluster structures at these elevated entropy levels indicates that the TF–IDF features are increasingly impacted by irrelevant or noisy data, which complicates distinguishing between categories. Thus, the PCA plots reveal that as entropy rises, the dataset's quality, in terms of category separation, diminishes. While lower entropy levels support the integrity of category boundaries, resulting in well-separated and distinct clusters, higher entropy levels pose challenges by diminishing the clarity of these distinctions. Thus, further feature engineering or the application of other dimensionality reduction techniques might be required at higher entropy levels to counteract the loss of category distinction.

4. Data gathering

As detailed in the previous sections, the SIEVE datasets were obtained from synthetic log generation starting from real logs. Therefore, the first step is to gather genuine logs from real systems. In [2], several proprietary datasets (DS) are used for training different NLP classification models due to the scarcity of publicly available datasets. As remarked in Section 2.1, the existing DS landscape is indeed mainly populated with instances tailored for the evaluation of Intrusion Detection Systems (IDS) and Intrusion Prevention Systems (IPS), such as the widely employed NSL-KDD [13] and CICIDS2017 [28]. These datasets focus on One Class Classification (OCC), classifying event logs as either "*normal*" or "*anomalous*" from the perspective of network traffic. While invaluable for IDS/IPS training, this type of classification fails to highlight the nuanced event categorization that SIEMs usually perform, which places the challenge within the domain of Multiple Classes Classification (MCC). The sources identified as suitable for SIEVE generation are the following:

- Loghub [5]: A comprehensive GitHub repository that offers an assortment of log datasets. This repository encompasses many sources, including supercomputers, applications, web servers, and standalone software instances.
- OSSEC [6]: An open-source intrusion detection system that collects log samples from different sources.
- Public Security Log Sharing Site [7]: A website dedicated to log sharing, originating from many systems, devices, and security appliances.

These sources were selected because they contain several log producers, are publicly available, and depict various events that commonly occur in organizations' networks. Thus, they are extremely valuable and relevant from an SIEM point of view. Table 1 summarizes the class distribution of the Combined Dataset (CDS), comprising 440 distinct log templates in 30 unique classes. These templates were selected through a manual process: initial datasets containing multiple raw, heterogeneous log entries (including non-SIEM-related events) were removed to keep only SIEM-relevant logs, as well as duplicate entries and nonactionable noise (e.g., routine system health checks). The final 440 templates represent a subset of high-fidelity log structures, validated by a panel of 3 cybersecurity experts with 5+ years of experience managing production SIEM systems. Each template source was then transformed to a masked templated string to generate synthetic variants (e.g., parameterized timestamps, IPs, user agents) to minimize the risk of introducing bias to the produced dataset. The experts used the Elastic Common Schema (ECS) classification guidelines [29] to standardize how events are labeled. ECS uses a hierarchical classification system based on multiple fields, among those fields, *category* represents broad, general-purpose classes. For example, all authentication-related events belong to the `authentication` category, regardless of whether the authentication succeeded or failed. The *action* field, on the other hand, provides more specific information about what exactly happened. For instance, a successful login attempt would have *category*: `authentication` and *action*: `authentication-success`. The *action* field

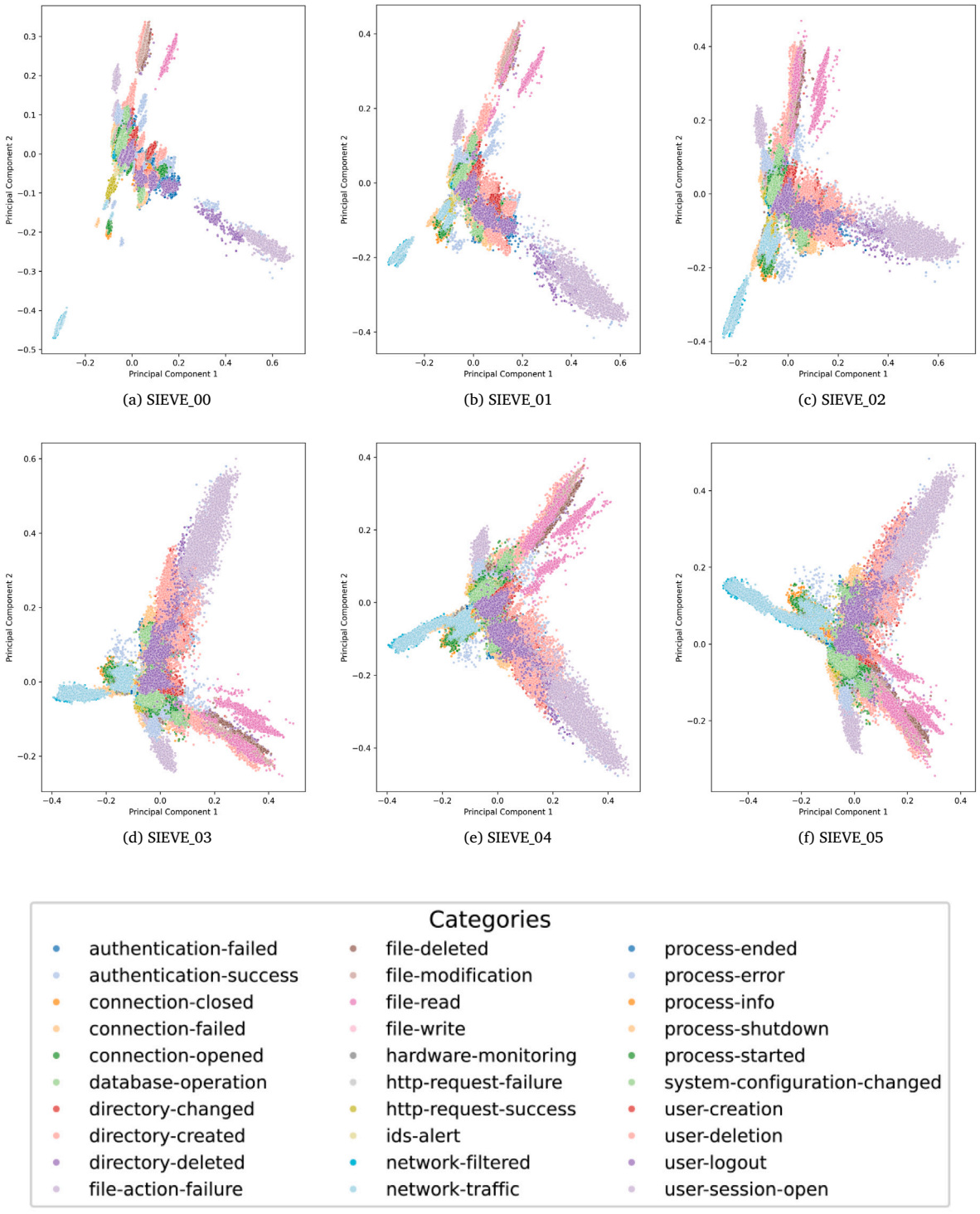


Fig. 2. Principal component analysis of TF–IDF plots of SIEVE.

is more important for SIEM systems because it enables more precise correlation rules and provides more detailed information for reports and visualizations. However, determining the correct *action* is more challenging than determining the *category* since it requires a more in-depth analysis of the semantic content of log messages. For example, while the word *"authentication"* is enough to classify a log into the authentication *category*, words like *"success"* or *"failure"* must be analyzed to determine the correct *action*, covering almost all of the ECS proposed classification categories. To obtain a general consensus and avoid conflicts, experts performed two rounds of blinded re-evaluation on 20% of randomly sampled templates, obtaining 0.82 Krippendorff's alpha score (Substantial agreement). Krippendorff's alpha [30] was selected over Cohen's kappa [31] due to its applicability to multi-rater scenarios and its robustness in handling multiple classes. As shown in Table 1, while the number of distinct classes (30) aligns with most datasets proposed in a similar multiclass classification study [2] (24, 25, 18), the limited number of log samples collected and the imbalance in class representations pose a challenge that must be addressed.

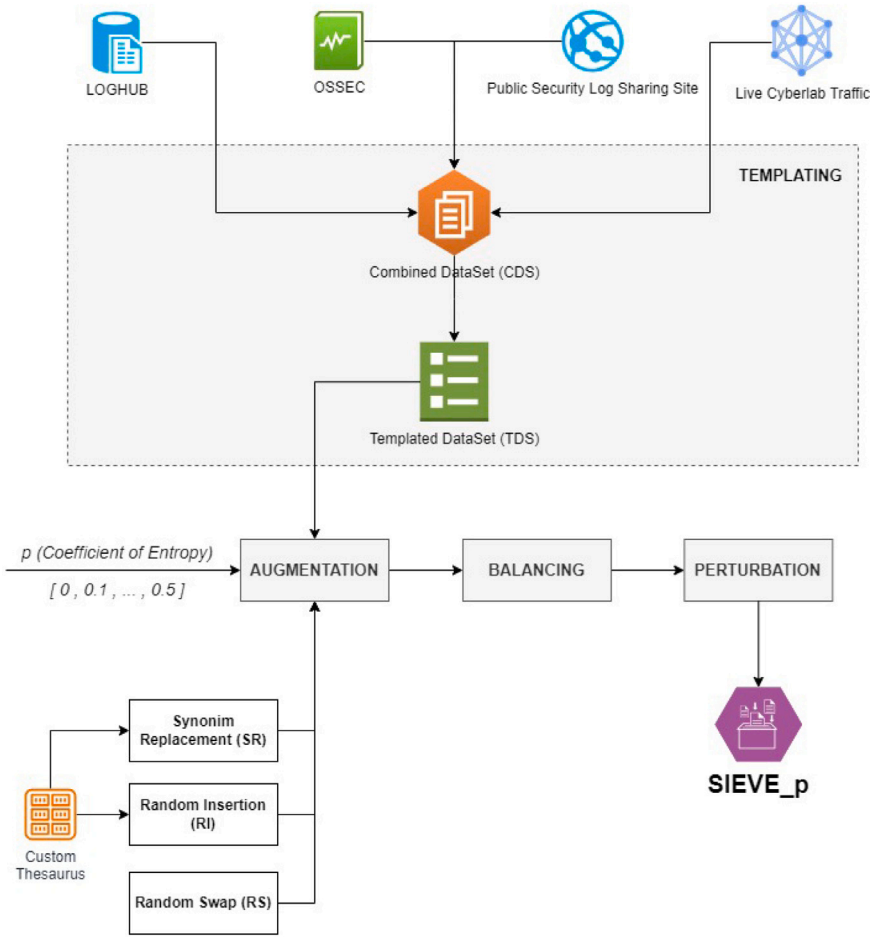


Fig. 3. SPICE process.

5. SPICE

The process designed to transform the CDS dataset into the SIEVE datasets is named *Semantic Perturbation and Instantiation for Content Enrichment* (SPICE) and involves several steps (shown in Fig. 3).

SPICE's stages are:

- **Templating**: each CDS sample is manually transformed into a generative template string by masking each variable token (IP addresses, hostnames, usernames, port numbers) with a placeholder, which is later replaced with a random consistent value. The use of placeholders allows for a structured approach to managing variable fields. Masking these variables initially ensures that the subsequent augmentation and balancing steps can focus on the core elements of the logs without being affected by the variability of these placeholders.
- **Augmentation**: The template strings generated in the previous step are augmented by a certain amount determined by the coefficient of the entropy p input parameter, using a combination of three techniques originally introduced by EDA [22]: Random Insertion (RI), Synonym Insertion (SR), and Random Swap (RS). Performing this step after placeholder masking is paramount because it naturally isolates the rest of the log message into strings containing semantically meaningful values that can be effectively augmented. This approach ensures that augmentation techniques can be applied to parts of the log message that contain relevant content.
- **Balancing**: An optimal distribution of generated samples per class is determined to achieve a balanced representation of unique classes in the output dataset. This calculation is based on the total number of samples desired for the output dataset;
- **Perturbation**: In this final phase, all placeholders inserted during the templating phase are replaced with randomized but contextually appropriate values. This step is crucial for maintaining the realism and integrity of SIEVE. For instance, a placeholder masking a username is substituted with another plausible username, while an IP address placeholder is replaced with a valid IP address. The perturbation process serves multiple important functions in such a methodology. Primarily, it ensures that the final synthetic logs closely resemble real-world scenarios. While these variable elements may not directly contribute to log classification, they play a vital role in log entries' overall structure and context. Moreover, this method enhances the quality of our training data. The presence of realistic, consistent variable elements helps create a more robust dataset, potentially leading to models that are better equipped to handle real-world data variations

Given the distinctive nature of log data content and the assessments outlined in the preceding sections, CheckList and EDA are the most suitable techniques for the task at hand. To elaborate further, it is possible to generate a potentially infinite number of event logs by introducing controlled perturbations and contextual randomizations to an extended list of tokens, exceeding what CheckListAugmenter [21] provides. These variations were created using the samples in the CDS as templates. However, relying solely on this approach may introduce bias because it would result in the repetitive use of terms excluded from the perturbation process. The perturbation process can be enhanced by incorporating three of the four methodologies introduced

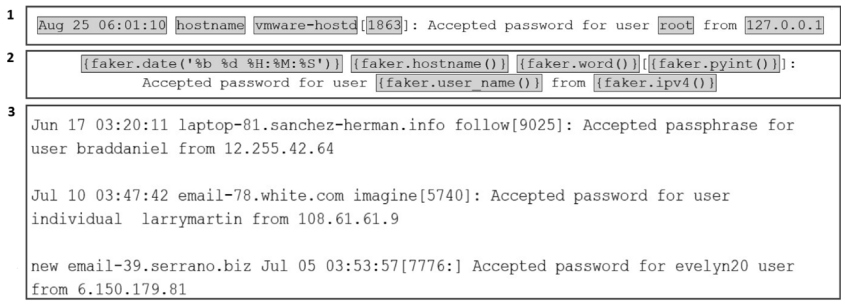


Fig. 4. SPICE augmentation steps.

in EDA (SI, RI, and RS) to address this challenge. This approach effectively combines two distinct methods and introduces SPICE as a novel method. When new data samples are generated, a certain level of randomness or uncertainty is added, referring to entropy in such cases. From the information theory perspective, entropy measures a system's unpredictability or disorder. Entropy, in this context, is used to increase the diversity of the dataset, which will affect the models through a better ability to generalize the unseen data. By introducing a controlled amount of entropy during data augmentation, models can avoid overfitting specific patterns and schemes in training data and become more robust to variations in real-world data. As is detailed in Section 5.2.2, the coefficient p acts by determining the number of transformations of type Synonym Replacement (SR), Random Insertion (RI), and Random Swap (RS) that will be applied to the log. Larger values of p result in more transformations, generating logs less similar to the originals in view of favoring generalization. Values less than p generate logs with few transformations, preserving the purity of the log. Several considerations regarding the choice p values are given in Section 3. The following sections detail each stage of the SPICE process.

5.1. Templating

The original Combined Dataset (CDS) consists of 440 entries, each containing a log message sample and its corresponding categorization label. These log samples, represented as character sequences, exhibit various features in terms of content and format. This phase aims to systematically examine and transform each log entry in the CDS through a manual, expert-guided process. This process involves several steps:

1. **Initial review:** The team individually examined each of the 440 log samples to understand their structure, content, and context within security information and event management (SIEM) systems.
2. **Identification of variable elements:** During the review, variable elements within each log sample were identified, such as IP addresses, user names, process IDs, and other technical indicators. These elements provide additional context but are not discriminating against log classification.
3. **Placeholder substitution:** Each previously identified variable element is replaced with a named placeholder. This placeholder can later be used to inject contextual perturbation relevant to the substituted string (e.g., replacing IP addresses with random IP addresses, not hostnames).

This manual process is necessary because each log represents a different device or application, which may have a unique structure that is challenging to identify automatically. While some tokens, such as IP addresses, timestamps, and hostnames, can be easily identified through automated methods, other more contextual information, such as usernames, must be manually identified within the log message to ensure consistent replacement.

Consider the log in Fig. 4 Step 1, which is retrieved from the OSSEC public resource. This log is a relevant message from the system security perspective, clearly identifying a successful authentication event. This type of event is labeled in the CDS as "`authentication_success`". All highlighted substrings in Fig. 4, Step 1 represents the aforementioned technical indicators that need to be replaced with placeholders to prepare the dataset for contextual perturbation using an interpretable structure, providing the added benefit of easier processing in subsequent steps, similar to what CheckListAugmenter does. The process of perturbing specific event sections is executed within the SPICE framework using the Faker library. This library has been chosen because of its effectiveness in similar case studies [32], as detailed in Section 5.4. The Faker library is a comprehensive collection of methods, also known as providers, each specialized in generating thematic textual data. Such generated data are context-dependent and include, but are not limited to, usernames, addresses, and hostages. This flexibility allows for a wide range of applications and use cases, making the Faker library a versatile data generation and manipulation tool. To streamline the perturbation process, during this stage, the placeholder that substitutes the event sections to be perturbed adheres to the ''`faker.<method_name>(<args>)`'' format. Here, "`method_name`" refers to the name of the Faker provider employed to generate perturbed data. The "`args`" are the provider function's input parameters, allowing for granular control over the resulting output string. This structured approach ensures consistent and controlled generation of perturbed data, thereby enhancing the reliability and effectiveness of the process. The output of this process, applied to the example log, is a string referred to as "log template" and is illustrated in Fig. 4, Step 2.

5.2. Augmentation

Applying only data perturbation techniques, particularly on variable tokens like IP addresses, usernames, and hostnames, raises concerns regarding their limited contribution to NLP classification. The inherent lack of specific information in these tokens increases the risk of being removed during data cleaning processes or entirely ignored by certain models. This underscores the need for a more comprehensive approach to guarantee the generation of diverse and meaningful data variants. To address these considerations, Easy Data Augmentation (EDA) [22] is applied. It is worth noting that this technique was initially introduced for a generic NLP-related text performance-boosting task and is therefore customized to fit the cybersecurity domain better, as explained in Section 5.2.2. The augmentation process involves two steps:

- **Tokenization**: during this step, each template log entry is deconstructed into atomic, indivisible sequences of characters;
- **Synthetization**: during this step, the adapted version of EDA is applied to eligible tokens.

```
Input: categories of the template sample
       p coefficient of entropy
       d dataset dimension
Output: SIEVE dataset containing d entries of the augmented sample

categories['templates'] <- loading template dataset
(1) total_instance_for_category, rest_c <- d/len(categories)

for category in categories:
(2)    total_instance_for_template,
       rest_t <- total_instance_for_category/len(category['templates'])

       for sample in templates:
              (3) tokens <- tokenize(sample)
              (4) augmented_versions <- modified_eda(tokens, total_instance_for_template, p)

              for augmented in augmented_versions:
                     joined_version <- join(augmented)
                     (5) perturbated <- perturbated(joined_version)

                     append_in_csv(category, perturbated)
```

Fig. 5. SPICE process pseudocode.

5.2.1. Tokenization

The first step of the augmentation process involves dissecting the template log into its fundamental components. These components are defined as atomic and indivisible sequences of characters, which are referred to as tokens. For clarity, tokens are categorized as follows:

- Templated sections enclosed within curly brackets;
- Spaces;
- Stop words;
- Any other alphanumerical sequences different from those mentioned above.

This initial step prepares for subsequent stages in which these tokens assume a central role. Furthermore, it provides an opportunity to identify groups of tokens that are particularly valuable in the log generation process. Such tokens, referred to as "Semantic Tokens" (ST), consist solely of alphabetical character sequences longer than two characters, which often carry semantic meaning useful for synonym evaluation and other linguistic tasks.

5.2.2. Synthetization

This step expands the number of logs by generating synthetic variants of the same sample by modifying the EDA approach. To investigate the specifics, EDA employs four distinct algorithms randomly chosen to create variations of generic sentences. These algorithms include Synonym Replacement (SR), Random Insertion (RI), Random Swap (RS), and Random Deletion (RD). Because random deletion (RD) may harm the accurate classification of log entries due to potential information loss, it was excluded from the set of algorithms used in this step. Each log entry may vary in length and include a different number of tokens. To introduce a homogeneous level of "noise", intended as an intentional alteration of existing data to create a more robust and diverse dataset collection, a "coefficient of entropy" (p) is introduced as a tuning parameter, ranging from 0 to 0.5. The reasons for the choice of this range have been explained in Section 3. This parameter is leveraged differently by each synthetization technique, but generally, the higher its value, the greater the alteration introduced:

- Synonym Replacement (SR): Replace n random ST words with synonyms defined in a thesaurus. In this case, n is the product of the meaningful tokens by the coefficient p.
- Random Insertion (RI): Insert a random synonym for a random ST in a random position. Repeat this process n times, where n is the product of ST by p.
- Random Swap (RS): Swap two non-space tokens. Repeat this operation n times, where n is the number of tokens times p.

When discussing RI and SR, a critical consideration arises regarding the choice of synonymous thesauri. When relying on the WordNet thesaurus, the issue of correctly replacing or inserting the synonyms of IT jargon-specific terms found in log data arises. This shortfall poses a potential threat to the dataset's semantic diversity. To address this problem, a customized thesaurus CYCUTHE (CYbersecurity CUstom THEsaurus), carefully tailored to conform to the nuances of IT jargon, is introduced. As a result, a more comprehensive and precise representation of the semantic landscape is provided, which addresses the potential limitations of a general-purpose thesaurus.

5.3. Balancing

Since the CDS is unbalanced, the process of new sample generation includes the following balancing procedure, aimed at obtaining a perfectly balanced dataset through equidistribution of classes and, within each class, equidistribution of templates.

Let C be the number of classes, S the total number of samples in the original dataset, and S_i the number of samples in class i (where $i = 1, 2, \ldots, C$). In this case, $C = 30$ and $S = 440$. To generate a balanced dataset with N new instances (e.g., $N = 100,000$), the number of instances N_i assigned to each class i should be equal to the ratio between N and C:

$$N_i = \frac{N}{C} \tag{1}$$

At this point, it is necessary to ensure that every single sample is represented equally within its reference category. To achieve this, the index j is introduced to represent the jth sample within category i. The number of augmented variants of the jth sample in category i is denoted by K_j:

$$K_j = \frac{N_i}{S_i} \tag{2}$$

where:

- N_i denotes the total number of samples in category i;
- S_i denotes the total number of original samples in category i.

The SIEVE balanced dataset generation pseudocode in Fig. 5 shows how the dataset is balanced by calculating the ratio of each category (1) and sample (2). Accordingly, each template goes through tokenization (3), augmentation with modified EDA (4) to generate multiple versions of the initial sample, and perturbation to create the final enhanced log (5), ready to be added with its proper category into the resulting dataset.

5.4. Perturbation

The outcome of the preceding steps ensures a substantial increase in the variety of log formats originating from the same sample. The final stage involves the reconciliation of previously isolated tokens within the enriched logs and replacing placeholders to introduce the necessary perturbations. This is achieved by generating random contextualized values using Faker. Table 2 illustrates a list of some of the most relevant providers offered by the Faker library, along with some customized providers that have been implemented during the experimentation phase to address specific use cases identified by (*).

The SPICE process creates multiple SIEVE datasets by setting different input parameter values for dataset size and entropy coefficients. When determining the dataset size, there are no specific criteria to define the optimal number of synthetic instances. The appropriate size may vary based on factors such as data representation, available computational resources, analytical goals, chosen classification models, and intrinsic features of the dataset. The first iteration of the dataset generation test involved the creation of a synthetic dataset comprising 35,000 samples. The dataset generation test is then repeated by increasing the dataset size to identify the limit size, resulting in negligible gains in terms of the trained classification model performance accuracy, as explained in Section 6.6. The same process is applied to the entropy coefficient p, progressively increasing it from 0 to 0.5 in 0.1 steps. Applying the generation process using such p value results in six distinct SIEVE_** datasets, evenly balanced across 30 classes. The asterisks denote varying entropy levels, with '00' signifying only perturbation without augmentation and '05' representing the highest level of augmentation entropy and perturbation. The next section details the experimentation process, providing insights into the methods employed.

6. Experiments

This section reports the experiments performed on SIEVE: different classification algorithms are benchmarked to show their performance against the proposed dataset. Using SIEVE, several experiments were carried out:

1. A benchmark has been performed to evaluate the performance of machine learning and deep learning algorithms on the SIEVE dataset. In this experiment, for each variant of the dataset, a portion of the dataset has been used for model training and the remaining portion for testing;
2. An additional experiment determined the dataset size by comparing the performance increase among the classifiers, considering three different versions of the datasets: the first of 35,000 items, the second of 50,000, and, finally, the version of 100,000.
3. Another experiment was carried out, evaluating the performance of the same classifiers for each variant of the dataset by testing on CDS. In this case, the entire variant of the SIEVE dataset has been used for training;

The third experiment evaluates the effectiveness of models trained on SIEVE datasets when applied to real logs. As detailed in Section 2, the SIEM events related dataset landscape presents a significant challenge: while numerous datasets exist, they predominantly focus on binary classification (normal/abnormal) rather than providing semantic labels for specific log types. CDS stands as the only available real log dataset for SIEM that employs a comparable semantic labeling taxonomy, making it uniquely suitable for validation and benchmark purposes.

The text classification task comprises several steps [33]:

1. Preprocessing: This step involves data cleaning, transformation, and feature engineering to enhance data quality and prepare it for modeling and analysis.
2. Training, Validation, and Testing: In this step, a portion of the generated data is used to train the models for each dataset variant. Validation assesses the models' performance on a reserved portion of the data while Testing further evaluates the models' classification capabilities using the original CDS dataset.
3. Evaluation: This step assesses the performance of all models using metrics like accuracy, precision, recall, and F1 score. It includes both validation data and the original CDS dataset used

Table 2
List of providers for faker.

Provider	Description
Bothify	Generate a string that contains numbers and letters, giving a placeholder as input
[a] camel_case_words	Random camel case sequence of words
Company	Random company name
Country	Random country name
Date	Random date with format
date_time	Get a datetime object for a date between January 1, 1970 and now
date_time_this_decade	Gets a object for the decade year
domain_name	Produce an Internet domain name with a specified number of subdomain levels
domain_word	Generate a single fake domain word
Email	Random email
file_name	Random file name
file_path	Random file path
Firefox	Generate a Mozilla Firefox web browser user agent string
Hexify	Generate a string with random hexadecimal characters
Hostname	Random generated hostname
http_method	Return a random method
[a] insert	Custom insert SQL script
ipv4	Random IPv4
ipv4_private	Returns a private IPv4
ipv6	Random ip v6
iso8601	Random date time in ISO 8601 format
Lexify	Generate a string with random characters
mac_address	Random mac address
md5	Random md5 id
Numerify	Sequence of numbers
Password	Random password
port_number	Random port number
Pyfloat	Random float number
Pyint	Random integer number
Pystr	Random string
[a] random_browser_agent	Randomly generate a browser user agent
random_choices	Generate a list of objects randomly sampled from the input elements
[a] random_dashed_words_number	Sequence of random words separated by dashes
random_element	Return a randomly sampled element
random_int	Generate a random integer between two integers
random_lowercase_letter	Generate a random lowercase ASCII letter
random_number	Generate a random integer
safe_domain_name	Return a safe domain
[a] select	Custom SQL script
Time	Obtain a time string with an optional imputable format
unix_device	Generate a Unix device file name
unix_time	Get a timestamp between January 1, 1970 and now, unless passed explicit start_datetime or end_datetime values
uri_page	Generate a fake page
uri_path	Generate a fake path
url	Generate a random url
user_name	Random username
uuid4	Random ID in uuid4 format
Word	Random word
words	Generate a tuple of words

[a] Custom made providers.

Table 3
Log preprocessing steps.

Step	Log	Description
1	*Jun 29 09:57:32 desktop-87.west.biz php-cgi: PHP Warning: PHP Startup: Unable to load dynamic library '/fact/tell/could/radio.avi' - /take/ten/arrive/nearly/social/class.wav: cannot open shared object file: No such file or directory in Unknown on line 2808*	Original Log
2	*"Jun", "29", "09:57:32", "desktop-87.west.biz", "php-cgi:", "PHP", "Warning:", "PHP", "Startup:", "Unable", "to", "load", "dynamic", "library", "'/fact/tell/could/radio.avi'", "–", "/take/ten/arrive/nearly/social/class.wav:", "cannot", "open", "shared", "object", "file:", "No", "such", "file", "or", "directory", "in", "Unknown", "on", "line", "2808"*	Tokenization
3	*"Jun", "29", "09:57:32", ":", "PHP", "Warning:", "PHP", "Startup:", "Unable", "to", "load", "dynamic", "library", "'/fact/tell/could/radio.avi'", "–", "/take/ten/arrive/nearly/social/class.wav:", "cannot", "open", "shared", "object", "file:", "No", "such", "file", "or", "directory", "in", "Unknown", "on", "line", "2808"*	IPs & URLs Removed
4	*":", "PHP", "Warning:", "PHP", "Startup:", "Unable", "to", "load", "dynamic", "library", "'/fact/tell/could/radio.avi'", "–", "/take/ten/arrive/nearly/social/class.wav:", "cannot", "open", "shared", "object", "file:", "No", "such", "file", "or", "directory", "in", "Unknown", "on", "line", "2808"*	Time removed
5	*"PHP", "Warning", "PHP", "Startup", "Unable", "to", "load", "dynamic", "library", "cannot", "open", "shared", "object", "file", "No", "such", "file", "or", "directory", "in", "Unknown", "on", "line", "2808"*	Paths & Punct Remove
6	*"PHP", "Warning", "PHP", "Startup", "Unable", "load", "dynamic", "library", "cannot", "open", "shared", "object", "file", "No", "file", "directory", "Unknown", "line", "2808"*	Stopwords Removed
7	*"php", "warning", "php", "startup", "unabl", "load", "dynam", "librari", "cannot", "open", "share", "object", "file", "no", "file", "directori", "unknown", "line", "2808"*	Stemming
8	*("php":10, "warning":2, "php":10, "startup":4, "unabl":5, "load":11, "dynami":2, "librari":10, "cannot":20, "open":19, "shared":12, "object":13, "file":25, "no":15, "file":25, "directori":19, "unknown":21, "line":23, "2808":1)*	TF-IDF Word Index

in testing. The evaluation quantifies the models' ability to learn from synthetic data produced by SPICE.

6.1. Preprocessing

It is important to mention that the preprocessing phase explained as follows is used only for the standard ML algorithms and it was not used for BERT architecture since the transformer architecture used in BERT has already its mechanism for tokenization and positional encoding for preprocessing. This preprocessing consists of four actions that transform the text into a suitable format for the ML algorithms:

- **Tokenization**: strings are split into words or smaller units for analysis or processing in natural language processing tasks, enabling text comprehension using the NLP text classification models detailed in Section 6.2.
- **Data cleaning**: common and low-semantic-content words and technical elements of the message, such as IP addresses and URLs, are removed from the text data. This improves the analysis and reduces noise in text-based classification.
- **Stemming**: To improve computational efficiency, words are reduced to their root or base form (text normalization) to group related words by removing suffixes.
- **TF–IDF and Normalization**: The TF–IDF technique is applied to manage text classification by weighing words against their importance in the text. Numeric features are normalized in the range [0,1].

The following sections detail the above steps.

6.1.1. Tokenization

Tokenization [34] is the process of splitting strings into smaller units, which can include splitting paragraphs into sentences and further dividing sentences into individual words. For example, consider the log in step 1 of Table 3, extracted from the SIEVE dataset. After tokenization, the log appears as in step 2. Tokenization is considered one of the most effective approaches [34] for preparing data for NLP processing. Breaking text into individual words simplifies the analysis of text data and streamlines the necessary steps before running machine learning algorithms.

6.1.2. Data cleaning

This phase eliminates tokens with poor or null semantic significance, which could otherwise hinder classifier models from accurate categorization. Such tokens include IP addresses, URLs, dates, file system paths, punctuation marks, and common stop words. It is crucial to note that this phase must precede stemming [35] to ensure the effectiveness of the text processing pipeline. The remainder of this section illustrates each step of the data cleaning process applied by SPICE on a tokenized sample log, as shown in Table 3. IP addresses, URLs, and punctuation marks are filtered through a regular expression (regex) [36]. Consider the tokenized log sample in step 2. After IP and URL removal, the message becomes as shown in step 3. Date and time signatures are removed in SPICE through datefinder [37,38]. To streamline this procedure, it is essential to reassemble the string to identify date patterns more effectively. Furthermore, the output can still be regarded as a list of tokens. After the date and time removal, the resulting sample log is shown in step 4. Subsequently, paths and punctuation are removed by applying a regex filter, and the resulting log is shown in step 5. The last data cleaning step is stopword removal. Stopwords are commonly occurring words such as "*and*", "*the*", "*in*", and "*to*" (in English), which are frequently found but have limited semantic value [39]. After the stopwords have been removed, the resulting log sample is shown in step 6.

6.1.3. Stemming

Stemming [35,40] is an NLP technique used to reduce tokens to their root or base form, called "*stem*". The primary purpose of stemming is to simplify words so that word variations are treated as the same word, even if they have different inflections or suffixes. In the context of a specific log classification task, stemming prevents the loss of semantic information. When two or more words share the same semantic meaning but differ in form, term frequency algorithms may face challenges in establishing the correct correlation. For instance, "*installation*" and "*install*" describe nearly identical actions, but no ML algorithm may readily recognize this similarity. Stemming is employed to reduce such words to a common root, such as "*install*", effectively addressing this issue. The Snowball Stemming Algorithm [41] has been used for experimentation because it supports multiple languages and has a lower error rate than other stemming algorithms. It is used in various libraries and tools that stem from different languages. Returning to the example, after stemming, the log appears as shown in step 7 in Table 3.

6.1.4. TF-IDF and normalization

Term-Frequency Inverse Document-Frequency (TF–IDF) [42–44] is a numerical statistic used to evaluate the importance of a word within a document relative to a collection of documents. Term Frequency (TF) measures a term's occurrence frequency in a document, and Inverse Document Frequency (IDF) defines its rarity across the entire dataset. Given $\mathrm{TF}(t, d)$ as the frequency of a single word inside a document, where t means term and d document and $\mathrm{DF}(t)$ as the number of

Table 4
LSTM input embedding size.

LSTM input embedding size	
$p = 0$	118,333
$p = 0.1$	137,643
$p = 0.2$	148,057
$p = 0.3$	155,892
$p = 0.4$	161,024
$p = 0.5$	164,839

Table 5
NLP model parameters.

NLP model parameters	
NB	`alpha=1.0; force_alpha=True; fit_prior=True; class_prior=True`
DT	`random_state=0; max_depth=400; criterion='gini'; splitter='best'; min_samples_split=2; min_samples_leaf=1; min_weight_fraction_leaf=0.0; max_features=None; max_leaf_nodes=None; min_impurity_decrease=0.0; class_weight=None; ccp_alpha=0.0; monotonic_cst=None`
KNN	`n_neighbors=300; weights='uniform'; algorithm='auto'; leaf_size=30; p=2; metric='minkowski'; metric_params=None; n_jobs=None`
SVM	`C=1.0; kernel='rbf'; degree=3; gamma='scale'; coef0=0.0; shrinking=True; probability=False; tol=0.001; cache_size=200; class_weight=None; verbose=False; max_iter=-1; decision_function_shape='ovr'; break_ties=False; random_state=None`
SGD	`tol=1e-3; loss='hinge'; penalty='l2'; alpha=0.0001; l1_ratio=0.15; fit_intercept=True; max_iter=1000; shuffle=True; verbose=0; epsilon=0.1; n_jobs=None; random_state=None; learning_rate='optimal'; eta0=0.0; power_t=0.5; early_stopping=False; validation_fraction=0.1; n_iter_no_change=5; class_weight=None; warm_start=False; average=False`
LR	`random_state=0; max_iter=300; penalty='l2'; dual=False; tol=0.0001; C=1.0; fit_intercept=True; intercept_scaling=1; class_weight=None; solver='lbfgs'; multi_class='auto'; verbose=0; warm_start=False; n_jobs=None; l1_ratio=None`
RF	`max_depth=50; random_state=0; n_estimators=100; criterion='gini'; min_samples_split=2; min_samples_leaf=1; min_weight_fraction_leaf=0.0; max_features='sqrt'; max_leaf_nodes=None; min_impurity_decrease=0.0; bootstrap=True; oob_score=False; n_jobs=None; verbose=0; warm_start=False; class_weight=None; ccp_alpha=0.0; max_samples=None; monotonic_cst=None`

documents in the document set that contain term t, IDF(t) is defined as follows:

$$\mathrm{IDF}(t) = \log \frac{1+n}{1+\mathrm{DF}(t)} + 1 \tag{3}$$

and

$$\text{TF-IDF}(t,d) = \mathrm{TF}(t,d) \times \mathrm{IDF}(t) \tag{4}$$

The resulting vectors are normalized by Euclidean normalization:

$$v_{norm} = \frac{v}{\|v\|_2} = \frac{v}{\sqrt{{v_1}^2 + {v_2}^2 + \ \ldots \ + {v_n}^2}} \tag{5}$$

Using TF–IDF, textual log entries are converted into a term-frequency inverse document frequency sparse matrix that provides a faster and memory-efficient approach, optimizing resource allocation. Applying TF–IDF to the example generates the word represented in Table 3 step 8.

6.2. Machine learning algorithms employed

Several machine learning algorithms are used to finalize the NLP text classification process. To provide a consistent benchmark for SIEVE dataset test results, the ML algorithms referenced in [2] are used, namely:

- Naive-Bayes (NB) [45];
- Decision Tree (DT) [46];
- K-Nearest Neighbors (KNN) [47];
- Logistic regression (LR) [48];
- Random Forest (RF) [49];
- Stochastic gradient descent (SGD) [50];
- Support Vector Machine (SVM) [51].

In addition to these traditional machine learning models, experimentation was extended to include more complex models, the Bidirectional Encoder Representation from Transformer (BERT) [52], which makes use of Transformer neural network architecture to process sequential data and Long Short-Term Memory (LSTM) [53] which is a type of Recurrent Neural Network (RNN) [54] that has been proved to be effective in text classification scenarios [55]. A distinguishing characteristic of BERT lies in its innovative approach to contextual understanding. Diverging from conventional models that process text unidirectionally, BERT implements a bidirectional strategy, comprehensively analyzing both preceding and succeeding words to establish context. This methodology is actualized by implementing a "masked language model" (MLM) pre-training objective. In this paradigm, approximately 15% of input tokens undergo random masking, subsequently challenging the model to predict these obscured elements based on the surrounding linguistic environment. LSTM, on the other hand, employs memory cells and gating mechanisms to manage information flow across sequences. These gates dynamically select which features to retain, update, or discard, enabling the model to capture dependencies in token sequences. Unlike standard RNNs, which struggle with gradient decay over long sequences, LSTM's architecture preserves gradient stability, making it a good fit for log classification, where sequential order and inter-word relationships are semantically meaningful. The following section provides an overview of the performance of these ML models when trained using the SIEVE datasets, along with the configuration and hyperparameters set for each model.

6.3. Experimental setup

During the experimentation phase, various technologies are used for data cleaning, visualization, and model development. The selected programming language is Python [56] through JupyterLab. Pandas [57] is used for data manipulation, NLTK [39] is used for data cleaning, and Scikit-learn [58] is used for machine learning model development. The classification algorithm parameters are selected through GRID search and are reported in Table 5. As detailed in [52], this work tests the BERT model (Bidirectional Encoder Representations from Transformers) architecture. The architecture in question involves a sophisticated framework that employs more complex self-attention processes to analyze an input sequence arranged in layers consisting of multiple transformer encoders stacked on top of one another. Each layer includes a multi-head attention sub-layer and a feed-forward neural network sub-layer, making it possible to process complex information. This architecture comprises 12 layers, 12 heads, and 768 hidden units. The training methodology implemented for BERT utilizes the Adam optimizer with weight decay (AdamW). During the fine-tuning phase for task-specific applications, such as text classification, a reduced learning rate (typically 2^{-5}) is employed to mitigate catastrophic forgetting of pre-trained knowledge. This fine-tuning process often entails the addition of a task-specific layer superimposed on the pre-trained model, enabling BERT to adapt to a diverse range of NLP tasks while leveraging its pre-existing knowledge base. The output layer of the model incorporates a SoftMax activation function, generating a probability distribution across the target classes, as documented in the literature [59]. Cross-entropy loss is employed as the objective function

Table 6
Technical specifications.

Technical specifications	
CPU	AMD EPYC 7252 (x2)
GPU	GPU NVIDIA A30 24 GB (x2)
HD	HPE 1.6 TB NVMe MU SFF BC U.3ST MV SSD
RAM	HPE 16 GB 2RX8 PC4-3200AA-R SMART KIT (x4)
OS	Ubuntu 20.04.6 LTS

for the training process to quantify the dissimilarity between true and predicted labels. The Adam optimizer, with a learning rate of 0.001, is selected for its effectiveness in this type of task. Training proceeds with a batch size of 64 samples. Notably, the model achieves its peak performance within the first epoch, demonstrating a rapid ability to discern relevant patterns in the data while avoiding overfitting. The implemented LSTM model, on the other hand, employs a hierarchical structure beginning with an embedding layer that transforms input tokens into dense vector representations. The embedding dimensionality is set to 50, with the vocabulary size increasingly adjusted based on the entropy level of the input data, ranging from 118,333 tokens at zero entropy to 164,839 tokens at 0.5 entropy as per Table 4. The architecture consists of a dual-layer LSTM network with 128 hidden units, processing the embedded sequences in a batch-first configuration.

The evaluation of the SIEVE classification process involves Training, Validation, and Testing. Initially, the dataset is partitioned using a 75:25 ratio for training and validation. In the Training phase, the models are trained using the 75% portion of the dataset. Following this, in the Validation phase, the trained models are tested using the remaining 25% of the dataset to evaluate its performance. The final step is the Testing phase. In this phase, the previously trained and validated models are further tested for classification using the original 440 logs that comprise the CDS dataset. It is important to note that the SPICE generation process does not include the original logs in the SIEVE collection. Therefore, this phase provides a unique opportunity to evaluate the models' capabilities of using synthetic logs for classifying previously unseen ones.

6.4. Hardware setup

Table 6 summarizes the hardware configuration used in the experiments.

6.5. Metrics

Micro F1 Average Score, Precision, and Recall metrics are used to evaluate the techniques and the time taken to perform the classification process. Micro F1 Average Score was selected as the primary metric because of its significance for weighted multi-class text classification task performance measurement [60] and its use in similar studies [2]. Micro-averaging involves treating all units collectively without considering potential variations between classes. Micro-Precision is calculated, for a generic class k, using the following formula:

$$MaP = \frac{\sum_{k=1}^{K} TP_k}{\sum_{k=1}^{K} TotalColumn_k} = \frac{\sum_{k=1}^{K} TP_k}{GrandTotal} \tag{6}$$

Instead, the Micro-Recall is obtained, for a generic class k, using the following formula:

$$MaR = \frac{\sum_{k=1}^{K} TP_k}{\sum_{k=1}^{K} TotalRow_k} = \frac{\sum_{k=1}^{K} TP_k}{GrandTotal} \tag{7}$$

Micro-average precision and Micro-average recall measures rely on the sum of the True Positives (TP). Finally, having the Grand Total is possible to define the Micro Average F1 score as follows:

$$MaF1 = \frac{\sum_{k=1}^{K} TP_k}{GrandTotal} \tag{8}$$

where TP_k is the True Positive number of prediction and k is a generic class. As shown by (6), (7) and (8) MaP, MaR and $MaF1$ assume the same value. For this reason, only $MaF1$ results are reported. The elapsed time measurement starts from model definition, followed by training on 75% of the dataset, and ends with the test on the remaining 25%.

6.6. Results and discussion

As illustrated in Table 7, the benchmark experiments conducted on SIEVE yield insightful results. BERT achieves the best results of all in terms of MaF1, except for the SIEVE_00 and SIEVE_01 datasets, where it is narrowly outperformed by the SVM and LSTM, which, however, are very slow. The LR model, while less computationally intensive than SVM, still requires a significant amount of time and ranks third in terms of $MaF1$. LSTM ranks first only in SIEVE_01 and is shown to be the second slowest model. Despite being the fastest, the NB model does not match the performance of the other models, ranking as the worst performer for $MaF1$ alongside the KNN model. Except for BERT, which shows itself as the best tradeoff between times and MaF1, SGD shows itself as a good tradeoff model, as it shows slightly lower MaF1 than SVM but performs 200 times faster.

Figs. 6–8 show the confusion matrices related to the top performer model (SVM), LSTM and BERT. Notably, the three models can adequately recognize all classes but show a slight deterioration as p increases. The three models tend to confuse class No. 8 (`directory-deleted`) with class No. 10 (`file-deleted`); this is evidently due to the similarity (even conceptual) between the logs. In particular, SVM and LSTM struggle to distinguish between them in all versions of SIEVE, classifying in about one out of four cases logs of type `file-deleted` with a label of `directory-deleted`; in this specific case, BERT behaves differently, showing problems in classification in the SIEVE_00, SIEVE_04 and SIEVE_05 datasets. Similarly, the models confuse logs related to class No. 6 (`directory-changed`) with logs related to class No. 11 (`file-modification`); again, the two types of logs are extremely similar to each other. However, unlike the previous case, SVM and BERT see a tendency for performance to deteriorate relative to classes 6 and 11 as p increases, while LSTM behaves differently, growing from $p = 0$ to $p = 0.2$, then returning to basis values in $p = 0.3$ and growing again to $p = 0.5$. The models also show some problems in correctly classifying the logs of class No. 28 (`user-logout`), which is confused with No. 2 (`connection-closed`); these logs, although similar, represent two quite different conceptual elements and, therefore, lead to particular effects on SIEM systems. In this case, for SVM and BERT the classification error increases as p increases, instead, LSTM is up and down. It should also be noted that there is a certain tendency for the models to confuse logs belonging to class No. 0 (`authentication-failed`) with class No. 1 (`authentication-success`). The latter circumstance is quite relevant in a SIEM system since potentially dangerous events, such as failed authentications, would be missed and misclassified as authentications that occurred correctly, which, on the other hand, represents equally important information. For example, in the case of brute force, it is important to understand whether there is one successful one following a large number of failed logins; such misclassification is particularly relevant in the context of the domain of interest.

Fig. 9 illustrates the comparative results of all experiments conducted on the SIEVE instance containing 100,000 samples for different entropy values. Generally, an increase in entropy corresponds to a progressive decrease in $MaF1$ performance. Considering that SPICE's synthetic generation method is designed to introduce controlled, semantically coherent perturbations that mimic natural log diversity while preserving fidelity, it is plausible to interpret this decrease of performance as a deliberate trade-off, useful to evaluate robustness against real-world log scenarios.

Table 7
Benchmark results on SIEVE (100,000 instances).

Model	SIEVE_00		SIEVE_01		SIEVE_02		SIEVE_03		SIEVE_04		SIEVE_05	
	MaF1	Time	MaF1	Time	MaF1	Time	MaF1	Time	MaF1	Time	MaF1	Time
NB	0.9335	00:00.36	0.9219	00:00.40	0.9098	00:00.39	0.9024	00:00.81	0.8946	00:00.70	0.8874	00:00.42
DT	0.9674	00:16.16	0.9522	00:24.40	0.9362	00:29.68	0.9279	00:33.98	0.9182	00:35.92	0.9090	00:37.07
KNN	0.9297	00:52.58	0.9297	00:50.72	0.9118	00:52.79	0.8975	00:53.50	0.8815	00:55.10	0.8373	00:58.10
LR	0.9720	01:47.58	0.9623	02:02.63	0.9500	02:24.61	0.9441	02:13.27	0.9339	02:26.98	0.9286	02:37.78
RF	0.9617	01:30.61	0.9390	01:46.21	0.9204	01:50.25	0.9061	01:53.89	0.8995	01:51.46	0.8943	01:57.83
SGD	0.9698	00:03.65	0.9582	00:05.02	0.9456	00:04.68	0.9371	00:03.32	0.9260	00:03.62	0.9218	00:04.27
SVM	0.9737	18:29.73	0.9626	10:30.29	0.9517	10:59.53	0.9458	11:33.46	0.9360	12:31.48	0.9323	13:11.97
BERT	0.9730	00:03:22	0.9624	00:03:32	0.9615	00:03:56	0.9554	00:03:58	0.9472	00:04:02	0.9528	00:04:19
LSTM	0.9715	05:10:00	0.9642	05:16:00	0.9465	05:15:56	0.9454	05:20:00	0.9212	05:33:03	0.9201	05:35:31

A quantitative analysis conducted across the proposed entropy levels demonstrates several key findings: First, at the baseline level ($p = 0$), all models achieve strong performance with MaF1 scores ranging from 0.9297 to 0.9737. Traditional machine learning models like SVM (0.9737), LR (0.9720), and SGD (0.9698) show particularly strong performance in a stable environment (total absence of entropy), while approaches like BERT (0.9730) and LSTM (0.9715) maintain competitive scores. As entropy increases, a gradual degradation pattern can be observed, with varying rates of decline: 1.03% ($p = 0 \rightarrow 0.1$), 1.28% ($p = 0.1 \rightarrow 0.2$), 0.91% ($p = 0.2 \rightarrow 0.3$), 1.16% ($p = 0.3 \rightarrow 0.4$), and 0.30% ($p = 0.4 \rightarrow 0.5$). This fluctuating rate suggests that models respond differently to increasing perturbation intensity. Advanced architectures like BERT show remarkable resilience with only a 2.08% total decrease from $p = 0$ (0.9730) to $p = 0.5$ (0.9528), indicating that SPICE's perturbations preserve relevant semantic features necessary for classification. At the highest entropy level ($p = 0.5$), the average MaF1 across all models remains at 0.9093 ($\pm$0.0334), with performance relatively distributed: six out of nine models maintain scores above 0.90. The most sensitive models – KNN (0.8373) and NB (0.8874) – maintain 89.1% and 95.1% of their original performance, respectively. This distribution suggests that SPICE's perturbations affect different algorithmic approaches with varying degrees. The relative stability is further confirmed by analyzing the coefficient of variation (CV) across entropy levels for each model. The maximum CV observed is 0.041 (KNN), with most models showing CVs below 0.030, indicating moderate performance volatility despite increasing levels of perturbation. BERT achieves one of the lowest CVs (0.008), demonstrating that transformer-based architectures can effectively leverage semantical variations.

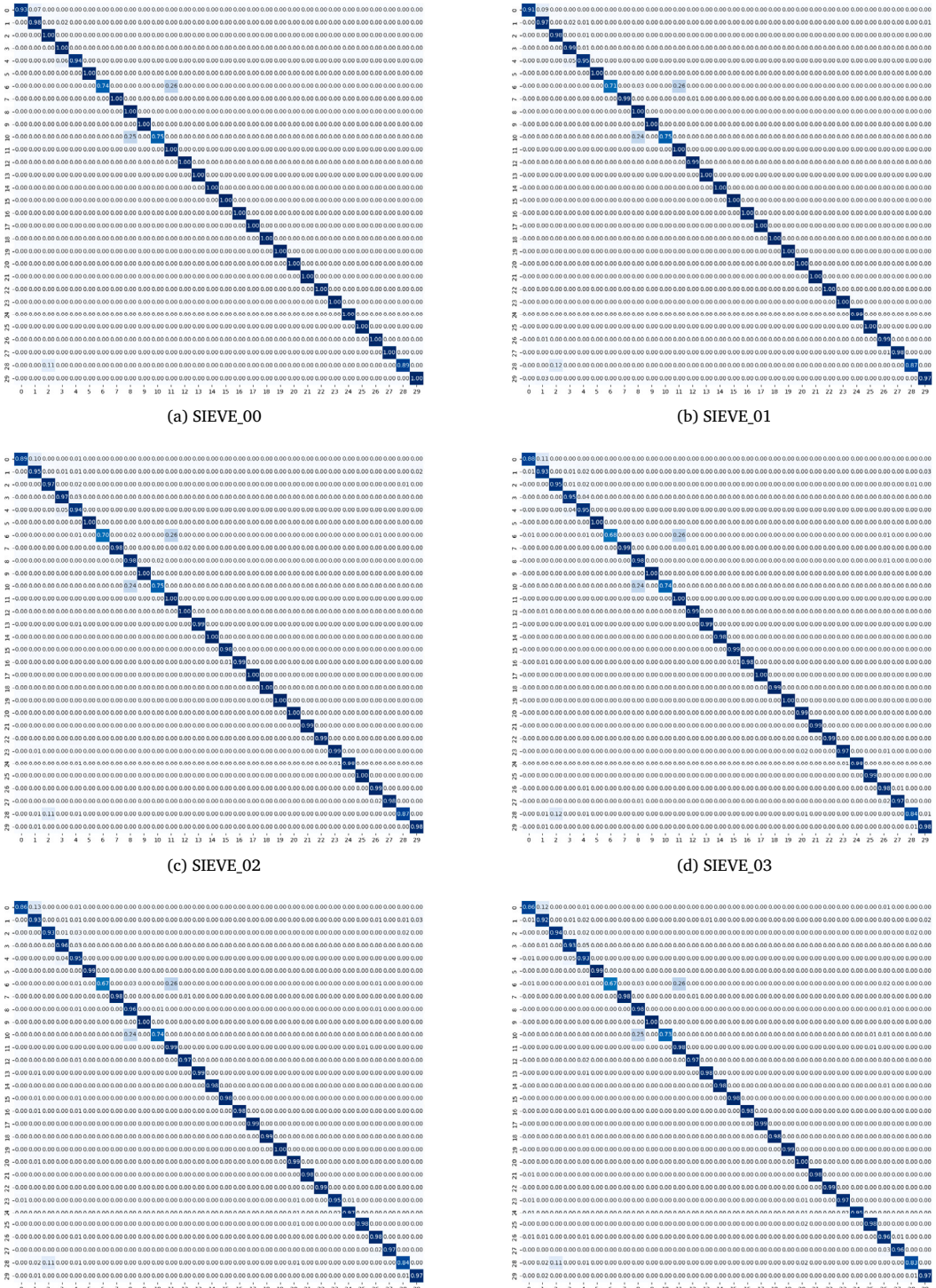
In addition to the quantitative evaluation via the MaF1 scores, we emphasize that the visual analyses provided by the KDE and PCA plots serve as an effective sensitivity study of the synthetic perturbations introduced by SPICE. Fig. 1 shows that the distribution of TF–IDF values remains largely consistent across different entropy levels, and Fig. 2 confirms that the overall arrangement of class clusters is maintained, even though slight overlaps are observed with increasing entropy. This convergence of evidence from visual inspections and the gradual trend observed in the MaF1 scores (Fig. 9) indicates that the impact of increasing synthetic perturbations is modest. Thus, our current evaluation framework, which integrates these methods, sufficiently captures the sensitivity of the models to entropy without the necessity to compute additional absolute or relative performance differences.

The results of the second experiment are now detailed. Tables 8 and 9 present benchmark results for datasets produced with SPICE, using total sample sizes of 50,000 and 35,000, respectively. These datasets are denoted as SIEVE_**_35k and SIEVE_**_50k, where ** represents the entropy coefficient ranging from 0 to 0.5. The purpose of these results is to identify the optimal dataset size. As indicated in Tables 7, 8, and 9, there is a significant average increase in accuracy values of 1.1184% when transitioning from the 35,000-sample dataset (with a mean accuracy of 0.9569) to the 50,000-sample dataset (with a mean accuracy of 0.9676). However, a relatively smaller average increase of 0.2690% is observed when moving from the 50,000-sample dataset to the 100,000-sample dataset (with a mean accuracy of 0.9702). These results suggest extending the dataset size beyond 100,000 samples will not significantly enhance accuracy.

Fig. 10 shows SVM, LSTM and BERT among those chosen for the SIEVE experiment for different entropy values and different sample sizes. The analysis revealsfor SVM and BERT a decrease in accuracy as data alteration increases, with a maximum variance of 0.06 across all dataset sizes. Despite this decline, the impact may be considered negligible. Notably, the decrease in accuracy is outweighed by the observed benefit of improved generalization for the trained model, particularly evident when increasing p. On the other hand, LSTM shows a diversified trend and is much more sensitive to variations.

Instead, Table 10 shows the results for the third experiment, showing the behavior of the models trained on SIEVE, tested on the real logs of the CDS dataset. The results reveal several important findings:

- The LR and SVM models consistently demonstrate superior performance across all entropy levels, with $MaF1$ scores consistently above 0.94. This is due to the capability of these models to maintain a balance between complexity and generalizability, effectively capturing the underlying structure of the data without overfitting to noise or specific patterns inherent in synthetic datasets, thus ensuring strong performance when applied to diverse and unseen real-world data.
- The SGD model follows closely behind the top performers, maintaining strong results with $MaF1$ scores ranging from 0.9113 to 0.9522 across all entropy levels. This showcases its effectiveness in balancing performance and computational efficiency.
- The DT and RF models show varying performance across entropy levels. The DT model achieves $MaF1$ scores between 0.8818 and 0.9363, while the RF model ranges from 0.8704 to 0.9181. This indicates sensitivity to changes in entropy levels and potential areas for optimization.
- The NB model exhibits consistent performance across entropy levels, with $MaF1$ scores hovering around 0.88. This suggests a stable generalization capability, albeit lower than the top-performing models.
- The KNN model continues to show the lowest performance among all models, with $MaF1$ scores ranging from 0.8431 to 0.8568. This reinforces the model's limitations for this particular task.
- The impact of increasing entropy levels on model performance varies across different algorithms. The LR and SVM models show remarkable resilience, with only slight decreases in performance as entropy increases. In contrast, models like DT and RF exhibit more pronounced drops. This suggests that the SPICE synthetic data generation process produces valuable training data across various entropy levels, with some models, particularly LR and SVM, being highly robust to these variations.



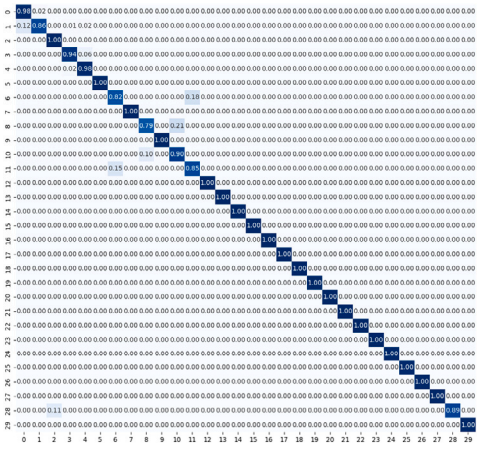
(a) SIEVE_00 (b) SIEVE_01

(c) SIEVE_02 (d) SIEVE_03

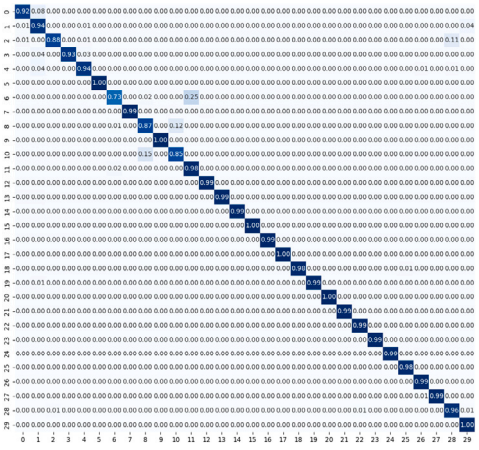
(e) SIEVE_04 (f) SIEVE_05

Fig. 6. SVM confusion matrices on SIEVE (100,000 instances) [x: predicted; y: actual].

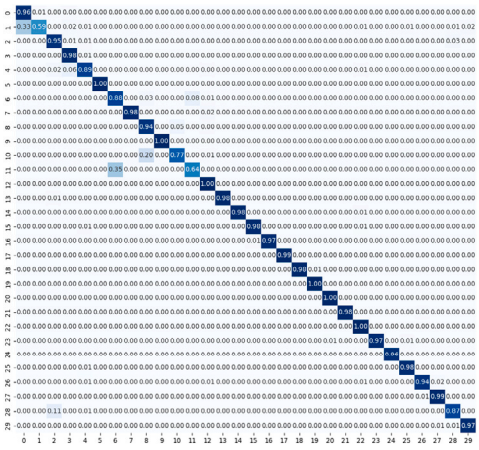
- BERT architecture outperforms all other techniques on the synthetic dataset, but its underperformance when tested on the original dataset can be partially attributed to its large number of parameters, which make it prone to learning noise present in synthetic datasets. Thus, overfitting occurs because BERT captures intricate, dataset-specific patterns that do not generalize well to the original log data, which might contain a slightly different pattern.
- The LSTM model demonstrates strong and relatively stable performance, with $MaF1$ scores ranging from 0.9042 to 0.9524, positioning it among the top-performing models. Its performance remains particularly robust at lower entropy levels ($p = 0$ to



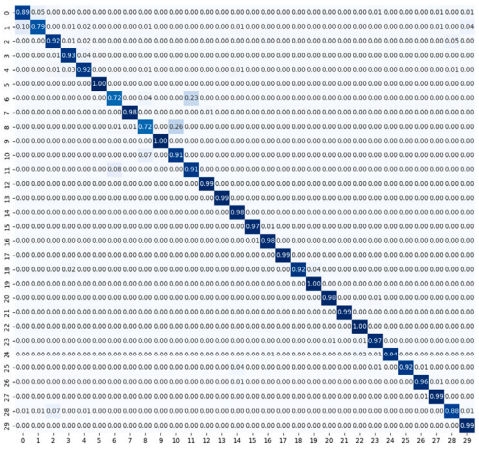
(a) SIEVE_00



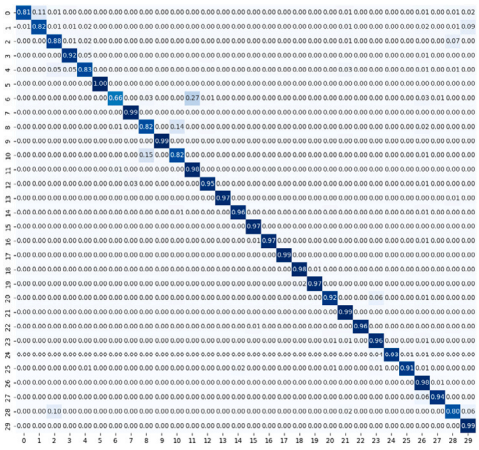
(b) SIEVE_01



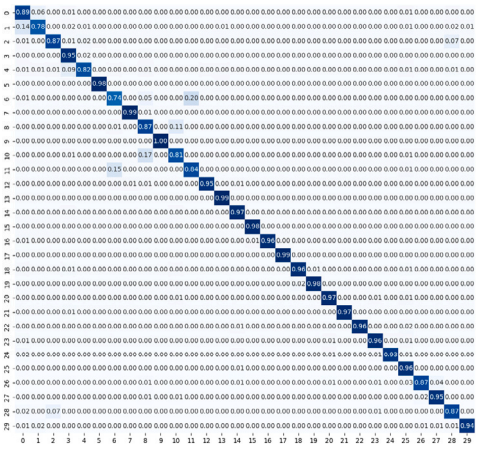
(c) SIEVE_02



(d) SIEVE_03



(e) SIEVE_04

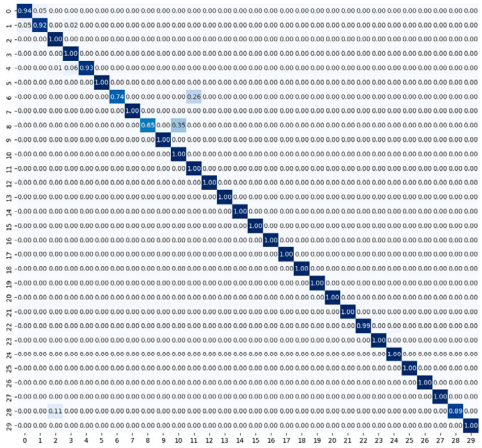


(f) SIEVE_05

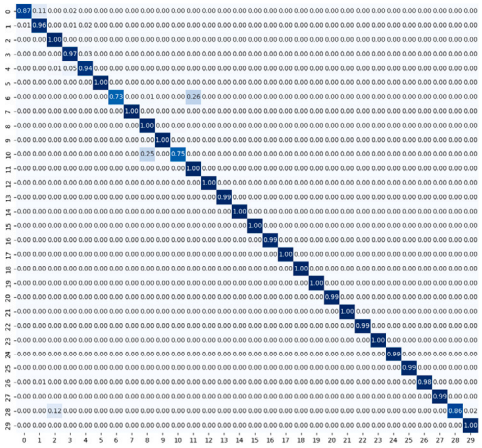
Fig. 7. LSTM confusion matrices on SIEVE (100,000 instances) [x: predicted; y: actual].

$p = 0.2$), maintaining scores above 0.95, with a notable decrease at $p = 0.3$ (0.9042) followed by a recovery at $p = 0.4$ (0.9426). Despite this variability, the LSTM's ability to maintain scores above 0.92 even at the highest entropy level ($p = 0.5$) indicates its potential for practical applications in log classification tasks.

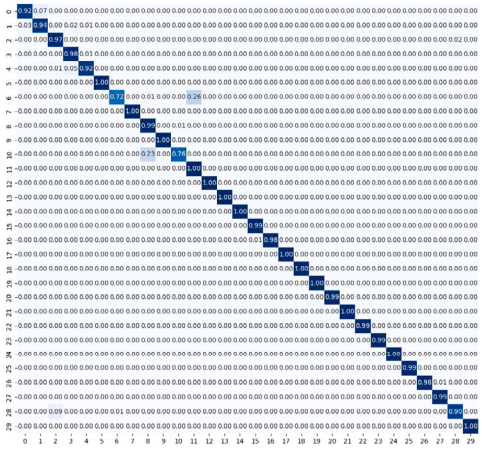
Figs. 11, 12, and 13 show the confusion matrices for this experiment of SVM, LSTM, and BERT. The test set consists of only 440 items, whose distribution for each class is shown in Table 1. In the case of the SVM, which, as already seen, maintains good performance in this experiment, the same classes already indicated in the first experiment



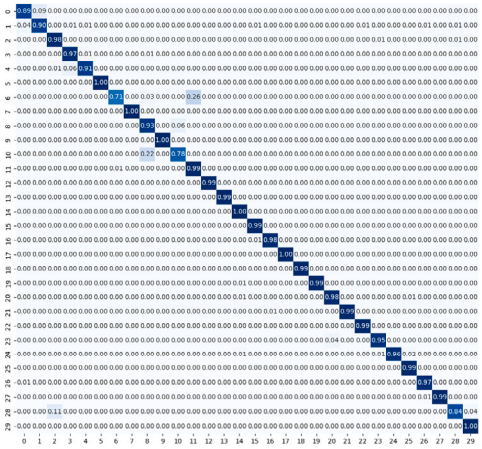
(a) SIEVE_00



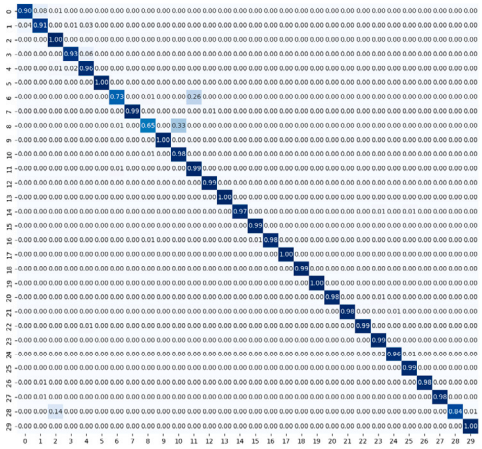
(b) SIEVE_01



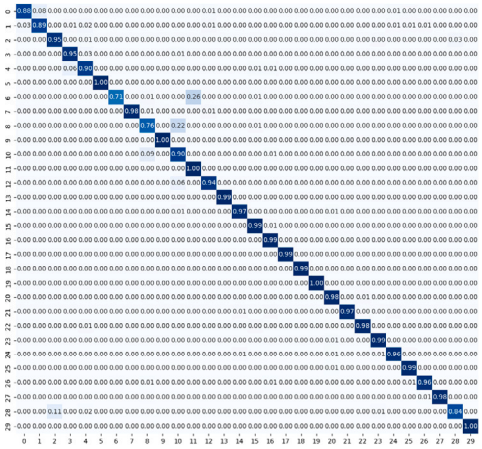
(c) SIEVE_02



(d) SIEVE_03



(e) SIEVE_04



(f) SIEVE_05

Fig. 8. BERT confusion matrices on SIEVE (100,000 instances) [x: predicted; y: actual].

are swapped between them. LSTM is shown to be much more sensitive in this experiment. For each dataset there are failures on different classes and, in particular, in the cases of SIEVE_03 and SIEVE_05 there are numerous misclassifications: the affected classes are the same as those referable to the first experiment, to which are added pairs 18 and 19 (`network-filtered` and `network-traffic`), 22 and 24 (`process-info` and `process-started`). BERT, however, in addition to the previously mentioned classes, there are differences in recognition errors for each dataset. BERT is shown to be more stable as p changes, but the classes affected by misclassification are different

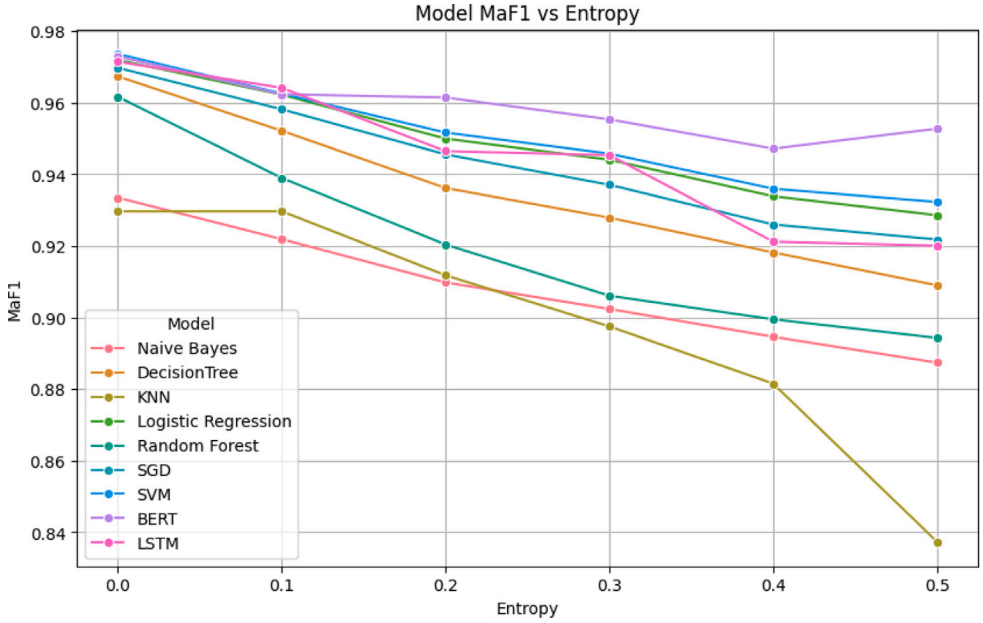


Fig. 9. Overall models performance on SIEVE collection (100,000 instances).

Table 8
Benchmark results on a reduced version of SIEVE (i.e., on 50,000 instances produced through SPICE, with the same mechanism concerning entropy).

Model	SIEVE_00_50k		SIEVE_01_50k		SIEVE_02_50k		SIEVE_03_50k		SIEVE_04_50k		SIEVE_05_50k	
	MaF1	Time	MaF1	Time	MaF1	Time	MaF1	Time	MaF1	Time	MaF1	Time
NB	0.9267	00:00.20	0.9120	00:00.20	0.9028	00:00.32	0.8928	00:00.33	0.8872	00:00.22	0.8782	00:00.22
DT	0.9693	00:06.81	0.9450	00:08.64	0.9276	00:10.44	0.9154	00:12.10	0.9039	00:12.84	0.8892	00:14.14
KNN	0.8975	00:24.81	0.8936	00:24.54	0.8888	00:24.29	0.8822	00:25.10	0.8762	00:17.81	0.8660	00:17.03
LR	0.9722	00:56.76	0.9593	01:03.17	0.9471	01:17.12	0.9378	01:19.10	0.9303	01:14.35	0.9221	01:15.83
RF	0.9644	00:37.78	0.9423	00:44.19	0.9255	00:48.57	0.9109	00:49.04	0.9069	00:49.18	0.8933	00:50.62
SGD	0.9724	00:01.09	0.9599	00:02.02	0.9480	00:02.03	0.9337	00:02.03	0.9297	00:02.08	0.9198	00:01.33
SVM	0.9743	06:09.68	0.9605	03:56.28	0.9529	03:55.00	0.9411	04:04.69	0.9341	04:21.53	0.9243	04:40.10
BERT	0.9710	00:02:04	0.9591	00:02:03	0.9568	00:02:26	0.9302	00:02:25	0.9387	00:02:40	0.9454	00:02:59
LSTM	0.9432	00:54:00	0.9420	00:58:00	0.9210	00:59:00	0.9205	01:00:00	0.9004	01:00:09	0.8982	01:01:10

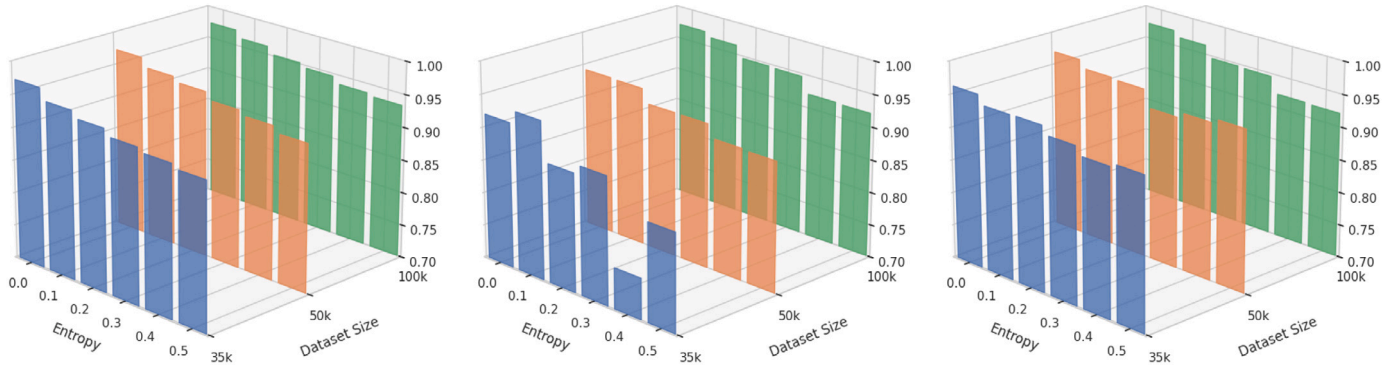
Table 9
Benchmark results on a reduced version of SIEVE (i.e., on 35,000 instances produced through SPICE, with the same mechanism concerning entropy).

Model	SIEVE_00_35k		SIEVE_01_35k		SIEVE_02_35k		SIEVE_03_35k		SIEVE_04_35k		SIEVE_05_35k	
	MaF1	Time	MaF1	Time	MaF1	Time	MaF1	Time	MaF1	Time	MaF1	Time
NB	0.9211	00:00.14	0.9054	00:00.22	0.9000	00:00.20	0.8841	00:00.15	0.8809	00:00.19	0.8754	00:00.15
DT	0.9651	00:03.68	0.9428	00:05.11	0.9237	00:06.37	0.9082	00:07.18	0.8971	00:07.88	0.8926	00:08.36
KNN	0.8560	00:13.95	0.8558	00:12.95	0.8549	00:14.35	0.8482	00:13.16	0.8436	00:13.37	0.8400	00:13.05
LR	0.9697	00:35.72	0.9575	00:40.05	0.9442	00:46.39	0.9310	00:46.31	0.9250	00:46.80	0.9185	00:46.90
RF	0.9579	00:24.73	0.9394	00:31.37	0.9261	00:30.57	0.9047	00:29.94	0.9002	00:32.87	0.8969	00:31.95
SGD	0.9702	00:00.75	0.9593	00:01.52	0.9453	00:01.49	0.9341	00:00.92	0.9262	00:01.62	0.9244	00:01.59
SVM	0.9737	03:10.22	0.9560	02:24.16	0.9460	02:17.60	0.9356	02:21.47	0.9293	02:36.21	0.9230	02:44.99
BERT	0.9639	00:01:39	0.9496	00:01:58	0.9505	00:02:11	0.9377	00:02:10	0.9248	00:02:29	0.9302	00:02:41
LSTM	0.9215	00:35:71	0.9410	00:35:95	0.8804	00:36:20	0.8939	00:36:40	0.7622	00:34:29	0.8487	00:37:10

from the previous ones: in addition to those referable to the first experiment, classes 11 and 25 (`file-modification` and `system-configuration-changed`); 18 and 20 (`network-filtered` and `process-ended`) are mixed up; classes 0, 1 (`authentication-failed`, `authentication-success`) are confused with 29 and 28 (`user-logout`, `user-session-open`). In light of this last experiment, SVM stands out as the best-performing model in log recognition. The test on real logs showed that SVM is more robust than LSTM and BERT in recognizing real logs due to the learning performed on SIEVE.

7. Conclusions

This research presents SIEVE, a collection of synthetic datasets for log text classification in SIEM-like environments. The datasets generated using the SPICE methodology aim to address data scarcity



(a) SVM (b) LSTM (c) BERT

Fig. 10. SVM, BERT and LSTM performance over varying SIEVE dataset sample size and entropy.

Table 10
Benchmark (MaF1) results of classifier models trained on growing entropy levels of the SIEVE (100,000 instances) Datasets and tested on the CDS dataset.

Model	SIEVE_00	SIEVE_01	SIEVE_02	SIEVE_03	SIEVE_04	SIEVE_05
NB	0.8909	0.8886	0.8886	0.8863	0.8886	0.8772
DT	0.9363	0.9068	0.9159	0.9159	0.8909	0.8818
KNN	0.8431	0.8522	0.8545	0.8545	0.8500	0.8568
LR	0.9681	0.9613	0.9613	0.9590	0.9522	0.9500
RF	0.9181	0.9113	0.9000	0.9022	0.8704	0.8795
SGD	0.9522	0.9409	0.9363	0.9318	0.9159	0.9113
SVM	0.9636	0.9636	0.9568	0.9590	0.9500	0.9477
BERT	0.9182	0.8864	0.9114	0.8795	0.8886	0.8864
LSTM	0.9524	0.9510	0.9505	0.9042	0.9426	0.9211

challenges in cybersecurity applications. Experimental results suggest that models trained on SIEVE can classify both synthetic and real-world log data with reasonable accuracy. The performance across varying levels of augmentation entropy indicates a degree of robustness in the dataset generation process. A gradual decrease in overall $MaF1$ is observed as the perturbation level increases systematically. However, this decline is marginal, indicating that the reduction in performance does not significantly undermine SIEVE's validity. In terms of model performance across all dataset variations, the SVM model emerged as the top performer, consistently delivering robust results across all dataset variations. BERT proved to be the most effective in terms of both $MaF1$ and time in testing on the same datasets, but, on the other hand, it failed to be robust in recognizing real logs from the CDS dataset. LR and SGD followed closely, demonstrating commendable performance. Conversely, KNN exhibited greater sensitivity to variations, making it more susceptible to changes in dataset configurations. LSTM showed good performance, but does not exhibit great stability as the dataset configurations change. Additional experiments testing models trained on SIEVE against the real-world CDS dataset were conducted to analyze the models' generalization capabilities further. Particularly, LR, SVM, and SGD showed strong generalization capabilities, maintaining high performance when classifying previously unseen logs, even with higher levels of entropy. This finding validates SIEVE's utility in addressing data scarcity challenges in NLP text log classification and its potential for practical applications in cybersecurity. Looking forward, these findings open several promising directions for future research. The application of advanced classification models, including deep learning architectures, transformer-based models, and neural networks, to leverage the synthetic datasets generated by SPICE presents an opportunity to uncover deeper insights, with the potential to expand the boundaries of what can be accomplished using synthetic data. Furthermore, exploring the impact of various augmentation strategies and techniques on these datasets may provide valuable insights into enhancing dataset diversity without significantly affecting accuracy. Additionally, investigating the performance of models trained on SIEVE across a wider range of real-world log datasets could further validate and extend the applicability of the proposed approach in multiple cybersecurity contexts.

CRediT authorship contribution statement

Pierpaolo Artioli: Writing – review & editing, Writing – original draft, Supervision, Methodology, Formal analysis, Data curation. **Vincenzo Dentamaro:** Writing – review & editing, Validation, Supervision, Methodology. **Stefano Galantucci:** Writing – review & editing, Writing – original draft, Methodology, Formal analysis, Data curation, Conceptualization. **Alessio Magrì:** Validation, Software, Investigation. **Gianluca Pellegrini:** Software, Methodology, Data curation. **Gianfranco Semeraro:** Validation, Investigation.

Generative AI

The authors declare that no generative AI was used to write this paper.

Funding

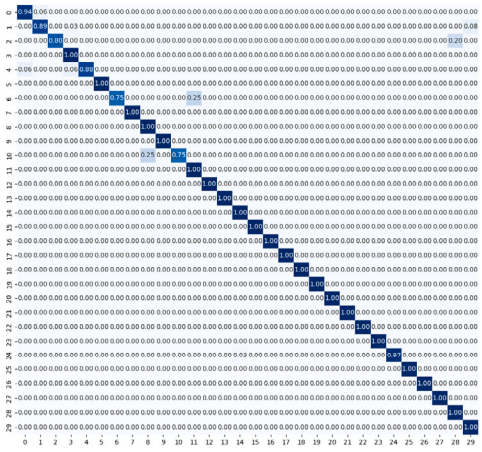
This work was supported in part by the Fondo Europeo di Sviluppo Regionale Puglia Programma Operativo Regionale (POR) Puglia 2014-2020-Axis I-Specific Objective 1a-Action 1.1 (Research and Development)-Project Title: CyberSecurity and Security Operation Center (SOC) Product Suite by BV TECH S.p.A., under Grant CUP/CIG B93G18000040007.

Declaration of competing interest

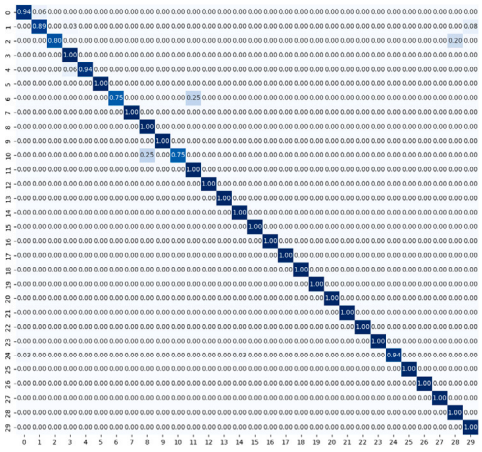
The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

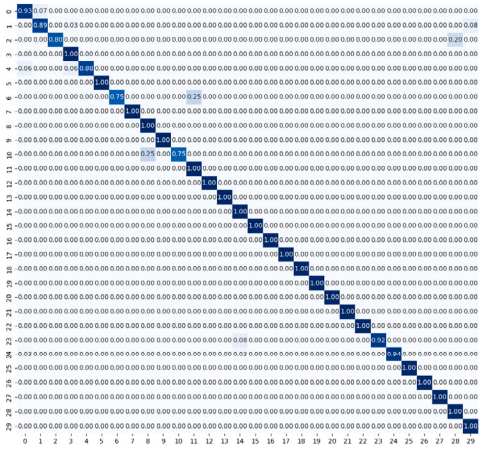
SIEVE Datasets are available here: https://www.cybersuite.it/en/sieve-cybersecurity-log-dataset-collection-forsiem-event-classification/.



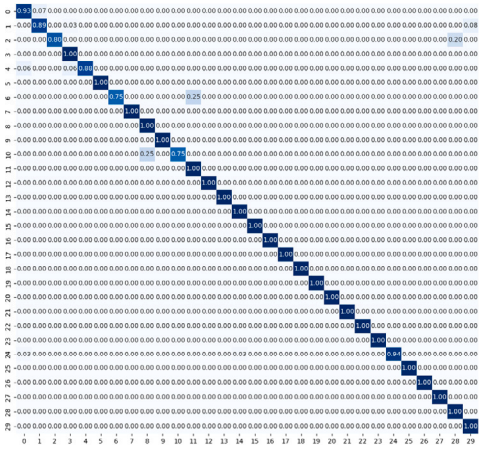
(a) SIEVE_00



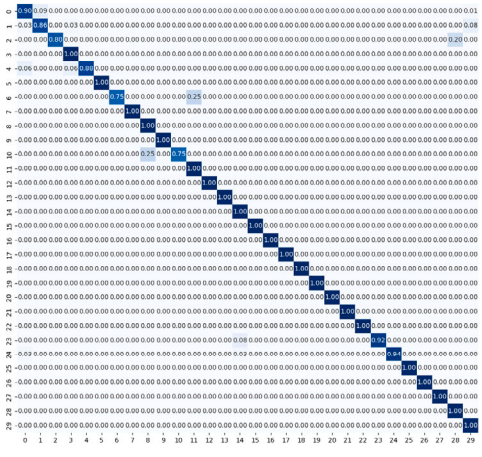
(b) SIEVE_01



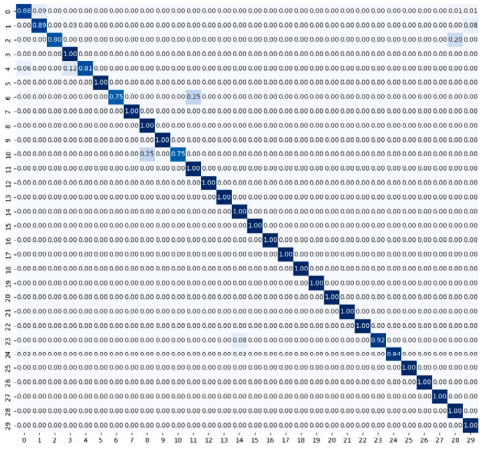
(c) SIEVE_02



(d) SIEVE_03

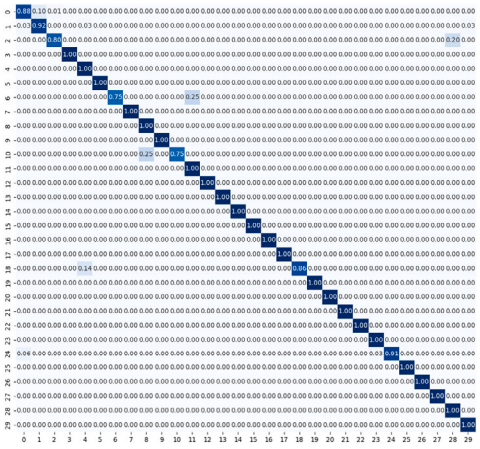


(e) SIEVE_04

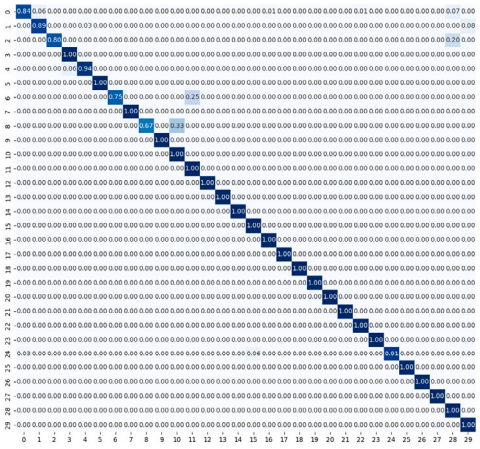


(f) SIEVE_05

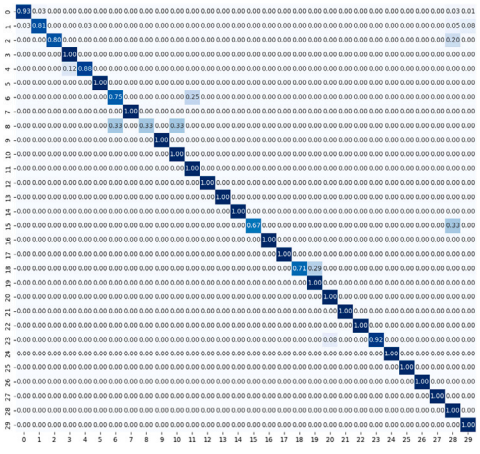
Fig. 11. SVM confusion matrices when trained on the different SIEVE datasets (100,000 instances) and tested on the CDS dataset [x: predicted; y: actual].



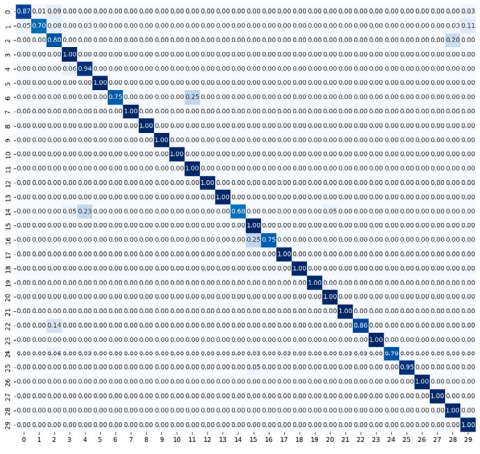
(a) SIEVE_00



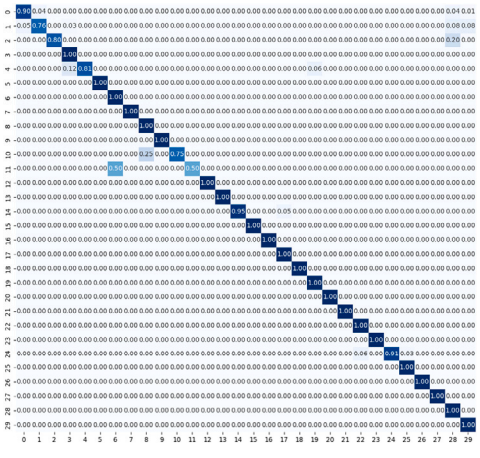
(b) SIEVE_01



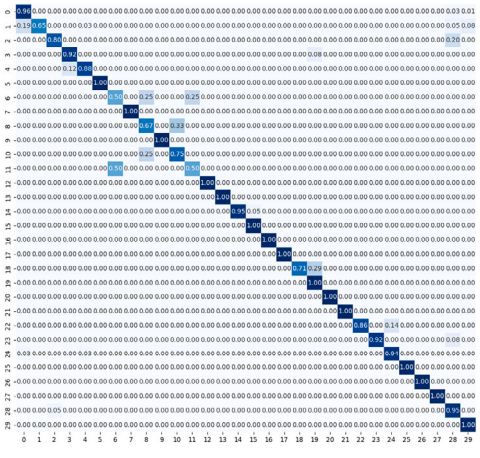
(c) SIEVE_02



(d) SIEVE_03

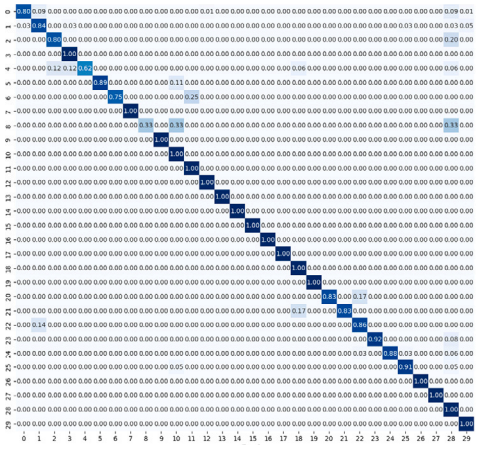


(e) SIEVE_04

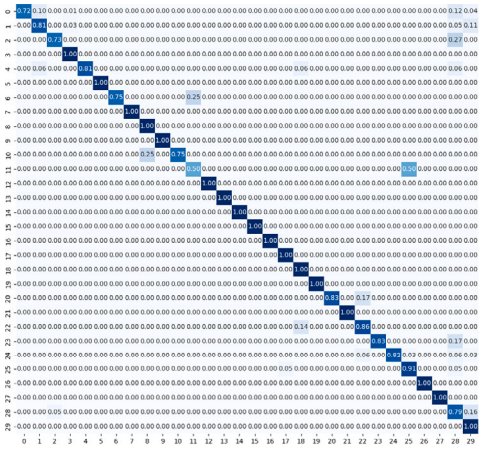


(f) SIEVE_05

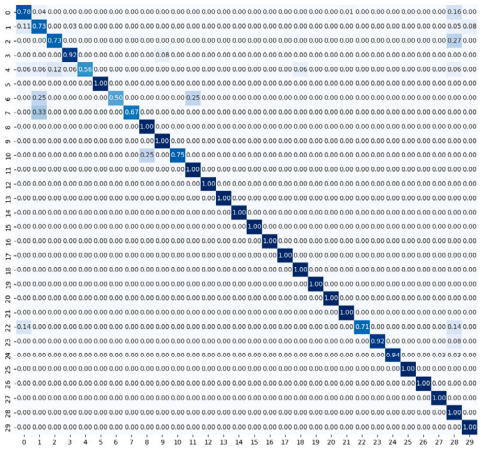
Fig. 12. LSTM confusion matrices when trained on the different SIEVE datasets (100,000 instances) and tested on the CDS dataset [x: predicted; y: actual].



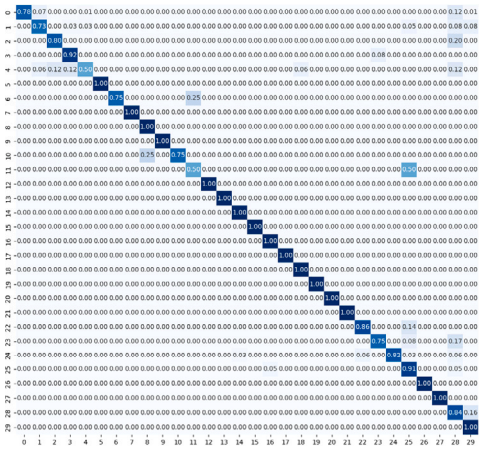
(a) SIEVE_00



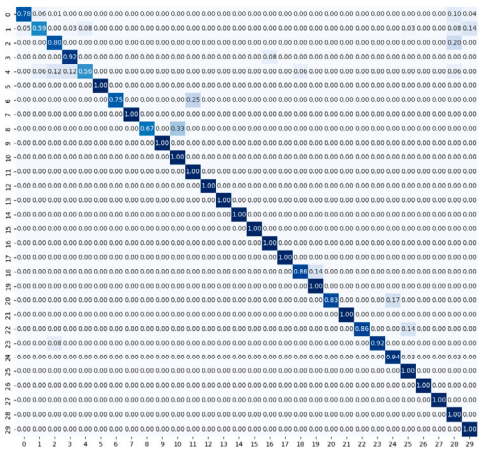
(b) SIEVE_01



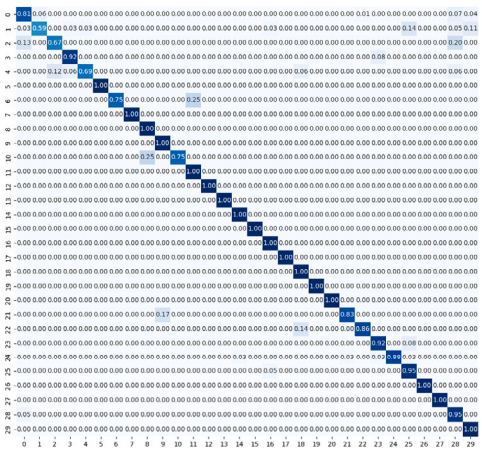
(c) SIEVE_02



(d) SIEVE_03



(e) SIEVE_04



(f) SIEVE_05

Fig. 13. BERT confusion matrices when trained on the different SIEVE datasets (100,000 instances) and tested on the CDS dataset [x: predicted; y: actual].

References

[1] G. González-Granadillo, S. González-Zarzosa, R. Diaz, Security information and event management (SIEM): analysis, trends, and usage in critical infrastructures, Sensors 21 (14) (2021) 4759.

[2] S.M. Hossain, R. Couturier, J. Rusk, K.B. Kent, Automatic event categorizer for SIEM, in: Proceedings of the 31st Annual International Conference on Computer Science and Software Engineering, 2021, pp. 104–112.

[3] I. Spasic, G. Nenadic, et al., Clinical text data in machine learning: systematic review, JMIR Med. Inform. 8 (3) (2020) e17984.

[4] M. Landauer, F. Skopik, M. Frank, W. Hotwagner, M. Wurzenberger, A. Rauber, Maintainable log datasets for evaluation of intrusion detection systems, IEEE Trans. Dependable Secur. Comput. 20 (4) (2022) 3466–3482.

[5] J. Zhu, S. He, P. He, J. Liu, M.R. Lyu, Loghub: A large collection of system log datasets for ai-driven log analytics, in: 2023 IEEE 34th International Symposium on Software Reliability Engineering, ISSRE, IEEE, 2023, pp. 355–366.

[6] OSSEC, OSSEC – World's Most Widely Used Host Intrusion Detection System (HIDS) 1.0 Documentation – Log Samples, https://www.ossec.net/docs/log_samples/.
[7] Public Security Log Sharing Site, https://log-sharing.dreamhosters.com/.
[8] R. Lippmann, J.W. Haines, D.J. Fried, J. Korba, K. Das, The 1999 DARPA off-line intrusion detection evaluation, Comput. Netw. 34 (4) (2000) 579–595.
[9] H.T. Nguyen, C. Torrano-Gimenez, G. Alvarez, S. Petrović, K. Franke, Application of the generic feature selection measure in detection of web attacks, in: Computational Intelligence in Security for Information Systems: 4th International Conference, CISIS 2011, Held at IWANN 2011, Torremolinos-Málaga, Spain, June 8-10, 2011. Proceedings, Springer, 2011, pp. 25–32.
[10] F. Zaker, Online shopping store - web server logs, 2019, http://dx.doi.org/10.7910/DVN/3QBYB5.
[11] M. Katara, Windows Event Log, URL https://www.kaggle.com/datasets/mehulkatara/windows-event-log/data.
[12] G. Mohi-ud din, NSL-KDD, 2018, http://dx.doi.org/10.21227/425a-3e55.
[13] M. Tavallaee, E. Bagheri, W. Lu, A.A. Ghorbani, A detailed analysis of the KDD CUP 99 data set, in: 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications, Ieee, 2009, pp. 1–6.
[14] A. Oyelakin, A. Ameen, T. Ogundele, T. Salau-Ibrahim, U. Abdulrauf, H. Olufadi, I. Ajiboye, S. Muhammad-Thani, et al., Overview and exploratory analyses of CICIDS 2017 intrusion detection dataset, J. Syst. Eng. Inf. Technol. (JOSEIT) 2 (2) (2023) 45–52.
[15] N. Moustafa, J. Slay, UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set), in: 2015 Military Communications and Information Systems Conference, MilCIS, IEEE, 2015, pp. 1–6.
[16] P. Bönninghausen, R. Uetz, M. Henze, Introducing a comprehensive, continuous, and collaborative survey of intrusion detection datasets, in: Proceedings of the 17th Cyber Security Experimentation and Test Workshop, 2024, pp. 34–40.
[17] D. Li, Y. Zhang, H. Peng, L. Chen, C. Brockett, M.-T. Sun, B. Dolan, Contextualized perturbation for textual adversarial attack, 2020, arXiv preprint arXiv:2009.07502.
[18] Q. Xie, Z. Dai, E. Hovy, T. Luong, Q. Le, Unsupervised data augmentation for consistency training, Adv. Neural Inf. Process. Syst. 33 (2020) 6256–6268.
[19] H. Dai, Z. Liu, W. Liao, X. Huang, Y. Cao, Z. Wu, L. Zhao, S. Xu, W. Liu, N. Liu, S. Li, D. Zhu, H. Cai, L. Sun, Q. Li, D. Shen, T. Liu, X. Li, AugGPT: Leveraging ChatGPT for text data augmentation, 2023, arXiv preprint arXiv:2302.13007.
[20] M.T. Ribeiro, T. Wu, C. Guestrin, S. Singh, Beyond accuracy: Behavioral testing of NLP models with CheckList, 2020, arXiv preprint arXiv:2005.04118.
[21] J.X. Morris, E. Lifland, J.Y. Yoo, J. Grigsby, D. Jin, Y. Qi, Textattack: A framework for adversarial attacks, data augmentation, and adversarial training in nlp, 2020, arXiv preprint arXiv:2005.05909.
[22] J. Wei, K. Zou, Eda: Easy data augmentation techniques for boosting performance on text classification tasks, 2019, arXiv preprint arXiv:1901.11196.
[23] W. Haider, J. Hu, J. Slay, B.P. Turnbull, Y. Xie, Generating realistic intrusion detection system dataset based on fuzzy qualitative modeling, J. Netw. Comput. Appl. 87 (2017) 185–192.
[24] J.-L. Wu, S. Huang, Application of generative adversarial networks and Shapley algorithm based on easy data augmentation for imbalanced text data, Appl. Sci. 12 (21) (2022) 10964.
[25] M. Ye, H. Wang, Z. Chen, MSMix: An interpolation-based text data augmentation method manifold swap mixup, 2023, arXiv preprint arXiv:2305.19617.
[26] Y. Chen, Y. Liu, NLI-based filtering for data augmentation in topic classification, in: 2022 IEEE/WIC/ACM International Joint Conference on Web Intelligence and Intelligent Agent Technology, WI-IAT, IEEE, 2022, pp. 103–110.
[27] T.M. de Barros, H. Pedrini, Z. Dias, Data-augmented emoji approach to sentiment classification of tweets, in: Iberoamerican Congress on Pattern Recognition, Springer, 2021, pp. 67–76.
[28] I. Sharafaldin, A.H. Lashkari, A.A. Ghorbani, Toward generating a new intrusion detection dataset and intrusion traffic characterization, ICISSp 1 (2018) 108–116.
[29] Elastic, Elastic Common Schema (ECS) Reference Documentation – Using the Categorization Fields, https://www.elastic.co/guide/en/ecs/current/ecs-using-the-categorization-fields.html.
[30] K. Krippendorff, Computing Krippendorff's alpha-reliability, 2011.
[31] J. Cohen, A coefficient of agreement for nominal scales, Educ. Psychol. Meas. 20 (1) (1960) 37–46.
[32] N. Gürsakal, S. Çelik, E. Biriş¸ci, Synthetic data generation with python, in: Synthetic Data for Deep Learning: Generate Synthetic Data for Decision Making and Applications with Python and R, Springer, 2023, pp. 159–214.
[33] A. Bhavani, B.S. Kumar, A review of state art of text classification algorithms, in: 2021 5th International Conference on Computing Methodologies and Communication, ICCMC, IEEE, 2021, pp. 1484–1490.
[34] S.J. Mielke, Z. Alyafeai, E. Salesky, C. Raffel, M. Dey, M. Gallé, A. Raja, C. Si, W.Y. Lee, B. Sagot, et al., Between words and characters: a brief history of open-vocabulary modeling and tokenization in nlp, 2021, arXiv preprint arXiv:2112.10508.
[35] M. Kantrowitz, B. Mohit, V. Mittal, Stemming and its effects on TFIDF ranking, in: Proceedings of the 23rd Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, 2000, pp. 357–359.
[36] J.A. Brzozowski, A survey of regular expressions and their applications, IRE Trans. Electron. Comput. (3) (1962) 324–335.
[37] A. Barbaresi, Htmldate: A Python package to extract publication dates from web pages, J. Open Source Softw. 5 (51) (2020) 2439.
[38] A. Koumjian, datefinder - extract dates from text (GitHub repository), https://github.com/akoumjian/datefinder#datefinder—extract-dates-from-text.
[39] S. Bird, NLTK: the natural language toolkit, in: Proceedings of the COLING/ACL 2006 Interactive Presentation Sessions, 2006, pp. 69–72.
[40] J.B. Lovins, Development of a stemming algorithm, Mech. Transl. Comput. Linguist. 11 (1–2) (1968) 22–31.
[41] A.G. Jivani, et al., A comparative study of stemming algorithms, Int. J. Comp. Tech. Appl. 2 (6) (2011) 1930–1938.
[42] F. Debole, F. Sebastiani, Supervised term weighting for automated text categorization, in: Proceedings of the 2003 ACM Symposium on Applied Computing, 2003, pp. 784–788.
[43] G. Salton, C. Buckley, Term-weighting approaches in automatic text retrieval, Inf. Process. Manage. 24 (5) (1988) 513–523.
[44] S. Senbel, Fast and memory-efficient TFIDF calculation for text analysis of large datasets, in: Advances and Trends in Artificial Intelligence. Artificial Intelligence Practices: 34th International Conference on Industrial, Engineering and Other Applications of Applied Intelligent Systems, IEA/AIE 2021, Kuala Lumpur, Malaysia, July 26–29, 2021, Proceedings, Part I 34, Springer, 2021, pp. 557–563.
[45] H. Zhang, The optimality of naive Bayes, Aa 1 (2) (2004) 3.
[46] W.-Y. Loh, Classification and regression trees, Wiley Interdiscip. Rev.: Data Min. Knowl. Discov. 1 (1) (2011) 14–23.
[47] J. Goldberger, G.E. Hinton, S. Roweis, R.R. Salakhutdinov, Neighbourhood components analysis, Adv. Neural Inf. Process. Syst. 17 (2004).
[48] E. Bisong, E. Bisong, Logistic regression, Build. Mach. Learn. Deep. Learn. Model. Google Cloud Platf.: Compr. Guid. Begin. (2019) 243–250.
[49] B. Xu, X. Guo, Y. Ye, J. Cheng, An improved random forest classifier for text categorization, J. Comput. 7 (12) (2012) 2913–2920.
[50] L. Bottou, Stochastic gradient descent tricks, in: Neural Networks: Tricks of the Trade: Second Edition, Springer, 2012, pp. 421–436.
[51] K. Kowsari, K. Jafari Meimandi, M. Heidarysafa, S. Mendu, L. Barnes, D. Brown, Text classification algorithms: A survey, Information 10 (4) (2019) 150.
[52] I. Turc, M.-W. Chang, K. Lee, K. Toutanova, Well-read students learn better: On the importance of pre-training compact models, 2019, arXiv preprint arXiv:1908.08962.
[53] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Comput. 9 (8) (1997) 1735–1780, http://dx.doi.org/10.1162/neco.1997.9.8.1735, arXiv:https://direct.mit.edu/neco/article-pdf/9/8/1735/813796/neco.1997.9.8.1735.pdf.
[54] J.J. Hopfield, Neural networks and physical systems with emergent collective computational abilities, Proc. Natl. Acad. Sci. 79 (8) (1982) 2554–2558, http://dx.doi.org/10.1073/pnas.79.8.2554, arXiv:https://www.pnas.org/doi/pdf/10.1073/pnas.79.8.2554, URL https://www.pnas.org/doi/abs/10.1073/pnas.79.8.2554.
[55] S. Jamshidi, M. Mohammadi, S. Bagheri, H.E. Najafabadi, A. Rezvanian, M. Gheisari, M. Ghaderzadeh, A.S. Shahabi, Z. Wu, Effective text classification using BERT, MTM LSTM, and DT, Data Knowl. Eng. 151 (2024) 102306, http://dx.doi.org/10.1016/j.datak.2024.102306, URL https://www.sciencedirect.com/science/article/pii/S0169023X24000302.
[56] G. Van Rossum, F.L. Drake, et al., Python Reference Manual, vol. 111, Centrum voor Wiskunde en Informatica Amsterdam, 1995.
[57] J. Reback, W. McKinney, J. Van Den Bossche, T. Augspurger, P. Cloud, A. Klein, S. Hawkins, M. Roeschke, J. Tratner, C. She, et al., Pandas-dev/pandas: Pandas 1.0. 5, Zenodo (2020).
[58] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al., Scikit-learn: Machine learning in Python, J. Mach. Learn. Res. 12 (2011) 2825–2830.
[59] T. Pearce, A. Brintrup, J. Zhu, Understanding softmax confidence and uncertainty, 2021, CoRR abs/2106.04972. arXiv:2106.04972. URL https://arxiv.org/abs/2106.04972.
[60] M. Grandini, E. Bagli, G. Visani, Metrics for multi-class classification: an overview, 2020, arXiv preprint arXiv:2008.05756.

Pierpaolo Artioli is a computer science and cybersecurity professional. He earned his bachelor's degree in computer science from the University of Bari in 2008. With experience in Defense and Banking, he currently works as a Cyber Security Software Architect Senior at BV Tech S.p.A. in Grottaglie, Italy. His research interests include Big Data, Cyber Security, and AI algorithms for User and Entity Behavior Analysis and Anomaly Detection.

Vincenzo Dentamaro is an assistant professor at the University of Bari. He received the degree in computer science from the Department of Computer Science, University of Bari, Italy, and the M.Sc. degree in machine learning from the Georgia Institute of Technology, Atlanta, GA, USA. He received a PhD in computer science from the University of Bari with a scholarship offered by InnovaPuglia S.p.A. He was a Software Engineer at Johnson Controls Inc.; an Intern at IBM Rome; and the CEO and CTO Nextome S.R.L. He is currently publishing in various pattern recognition journals and conferences. He is associated editor for the Springer-Nature journal Multidimensional Systems and Signal Processing. He is also a Reviewer of IEEE Access, Elsevier Pattern Recognition Journal, MDPI Sensor, MDPI Information, and so on. He has previously published about indoor positioning and localization techniques on Microsoft Research Journal and holds two international patents on localization technologies. His awards and honors include the 1st prize Busan Metropolitan City, South Korea, in the Seal of Excellence European Commission; IBM's Global Mobile Innovator Tournament Award at the Mobile World Congress; and the MIT Technology Review award.

Stefano Galantucci is a Research Fellow at the Computer Science Department, University of Bari Aldo Moro, Italy. In 2024 he received a PhD degree in Computer Science, defending a thesis in cybersecurity concerning innovative methodologies for Firewalls and Intrusion Detection/Prevention Systems. He also holds a MSc degree in Cybersecurity with full marks and honors. He has published various scientific papers in international journals and conferences on topics covering the research areas of cybersecurity, network and application security, cryptography, and biometrics. He is also an adjunct lecturer teaching Cryptography at the Master's Degree in Cybersecurity at the University of Bari Aldo Moro. He is also a member of multiple examination boards at the University of Bari Aldo Moro, including Biometric Systems (MSc in Cybersecurity) and Models and Methods for Application Security (BSc in Computer Science). He is a Reviewer for many Journals, including IEEE Transactions on Information Forensics and Security, and Computers & Security.

Alessio Magrì received the M.Sc. degree (Hons.) in Cybersecurity from the "Aldo Moro" University of Bari, Italy, in 2022. During his studies, he developed skills and knowledge on Cyber Security, Artificial Intelligence and Network Security, working firstly as Cybersecurity Consultant and currently as a Cybersecurity Software Developer for the R&D Cyber Lab of BV TECH S.p.A, Grottaglie, Italy. His main research interests include Penetration Testing, Artificial Intelligence algorithms for Network Security and SIEMs.

Gianluca Pellegrini, a Software Developer with a Bachelor's degree in Computer Science from the "Aldo Moro" University of Bari, Italy, in 2012. Has over a decade of experience in the tech industry. Working as software developer for the R&D Cyber Lab of BV TECH S.p.A.

Gianfranco Semeraro received a Bachelor's degree in Computer Science and a Master's degree in Data Science from the Department of Computer Science, University of Bari. He obtained his National PhD in Sustainable Development and Climate Change, Curriculum 3 - Technology and Territory at the IUSS University of Pavia and the University of Bari Aldo Moro. He conducts research in the field of computer science, in particular in handwriting processing for the identification of neurodegenerative diseases from early symptoms, in the analysis of spatial data on the socio-economic-environmental situation, optimisation of appliance scheduling based on electricity produced from renewable sources and analysis and prediction of events from event-log and in the field of Process Mining. He is also a reviewer for IEEE ACCESS, Pattern Recognition Letters, SN Computer Science and Expert Systems with Application.